

*Logo ESPRIT
(à insérer)*

2025 - 2026

PROJET DE FIN D'ÉTUDES

DIPLÔME NATIONAL D'INGÉNIEUR

SPÉCIALITÉ : TWIN

**Plateforme AIOps
Analyse Intelligente des Pipelines CI/CD**

Réalisé par : Khalil Ben Yahia Wenich

Encadrant ESPRIT : Mme Houda Jendoubi

Encadrant Entreprise : Mr [Nom de l'encadrant]

*Logo Capgemini
Altran Telnet Corporation*

Bandeau accréditations (UNESCO – CDIO – EUR-ACE – Cti – CGE)

Dedication

To my dear parents, for their unwavering support and sacrifices,

To my family and friends, for their constant encouragement,

To everyone who contributed, directly or indirectly, to the completion of this work.

I dedicate this modest achievement.

Acknowledgments

At the end of this final year project, I would like to express my deep gratitude to everyone who contributed to its success.

I first thank my academic supervisor, **Mrs. Houda Jendoubi**, for her guidance, valuable advice and availability throughout this work.

I also extend my sincere thanks to my company supervisor at **Capgemini — Altran Telnet Corporation Tunisia**, as well as to the whole engineering team, for their welcome, their trust and the sharing of their expertise on CI/CD practices and production-grade DevOps tooling.

My thanks also go to the faculty and administrative staff of **ESPRIT School of Engineering** for the quality of the training provided throughout my studies.

Finally, I thank the members of the jury for agreeing to evaluate this work.

Abstract

This final year project, carried out at **Capgemini — Altran Telnet Corporation Tunisia**, consists of designing and developing an **AIOps platform** (Artificial Intelligence for IT Operations) for the automated analysis of CI/CD pipelines.

The solution receives GitLab events in real time through secure webhooks, collects quality (SonarQube) and security (Trivy) reports, then submits the aggregated context to a Large Language Model (Groq — Llama 3.3 70B) to produce a root-cause analysis, a risk score and prioritized fix suggestions — all within ten seconds. The platform also provides incident management, a self-feeding knowledge base, a per-project health score (grade A to F) and weekly aggregated reports, presented in a real-time interactive React dashboard. It is containerized with Docker, deployed on Kubernetes (k3s) through Helm, and equipped with a complete Jenkins CI/CD pipeline featuring automated tests.

Keywords: AIOps, CI/CD, Root-Cause Analysis, LLM, GitLab, Kubernetes, Jenkins, Node.js, React.

Résumé

Ce projet de fin d'études, réalisé au sein de **Capgemini — Altran Telnet Corporation Tunisie**, consiste en la conception et le développement d'une **plateforme AIOps** pour l'analyse automatisée des pipelines CI/CD. La solution reçoit les événements GitLab en temps réel via des webhooks sécurisés, collecte les rapports de qualité (SonarQube) et de sécurité (Trivy), puis soumet l'ensemble à un grand modèle de langage (Groq — Llama 3.3 70B) pour produire une analyse de cause racine, un score de risque et des suggestions de correction priorisées, le tout en moins de dix secondes. La plateforme intègre également la gestion des incidents, une base de connaissances auto-alimentée, un score de santé par projet (grade A à F) et des rapports hebdomadaires, présentés dans un tableau de bord React temps réel. Elle est conteneurisée avec Docker, déployée sur Kubernetes (k3s) via Helm, et dotée d'un pipeline CI/CD Jenkins complet.

Mots-clés : AIOps, CI/CD, analyse de cause racine, LLM, GitLab, Kubernetes, Jenkins, Node.js, React.

Contents

Dedication	i
Acknowledgments	ii
Abstract	iii
List of Abbreviations	x
General Introduction	1
1 General Context and Problem Statement	2
1.1 Introduction	2
1.2 Host Organization	2
1.2.1 Organizational positioning	2
1.2.2 Stakeholder analysis	3
1.2.3 SWOT analysis of the initiative	5
1.3 Study of the Existing Situation	5
1.4 Critique of the Existing Situation	6
1.5 Proposed Solution	7
1.6 Working Methodology	8
1.7 Conclusion	9
2 State of the Art and Comparative Study	10
2.1 Introduction	10
2.2 Commercial AIOps and Observability Platforms	10
2.2.1 Datadog and Dynatrace	10
2.2.2 Moogsoft and BigPanda	10
2.2.3 PagerDuty / Jeli and incident-management tools	11
2.3 CI/CD-Native Tooling	11
2.4 Academic Background: LLMs for Log Analysis and RCA	11
2.5 Comparative Synthesis	12
2.6 Positioning of the Contribution	12
2.7 Conclusion	13

3	Requirements Analysis and Specification	14
3.1	Introduction	14
3.2	Actor Identification	14
3.3	Functional Requirements	14
3.3.1	Dashboard	14
3.3.2	Pipeline management	15
3.3.3	AI analysis — Root-Cause Analysis	15
3.3.4	Security and code quality	15
3.3.5	Knowledge base	15
3.3.6	Health score, incidents and reports	16
3.4	Non-Functional Requirements	16
3.5	Functional Behavior: Activity Diagram	17
3.6	Use-Case Modeling	18
3.6.1	Use-case refinement	19
3.7	Requirements Traceability Matrix	23
3.8	Conclusion	24
4	Design and Architecture	25
4.1	Introduction	25
4.2	Overall Architecture	25
4.2.1	Component diagram	26
4.2.2	Physical deployment diagram	26
4.3	Backend Architecture	27
4.4	Main Processing Flow	27
4.5	Data Models	28
4.5.1	Service-layer class diagram	31
4.6	Business Services	32
4.6.1	Health-score service	32
4.6.2	Knowledge-base service	32
4.6.3	Other services	32
4.7	REST API Design	33
4.7.1	API design rationale	33
4.7.2	Database design rationale	34
4.8	Additional Behavioral Views	34
4.9	Frontend Architecture	35
4.10	Conclusion	36
5	Implementation, Testing and Deployment	37
5.1	Introduction	37
5.2	Working Environment	37

5.2.1	Hardware environment	37
5.2.2	Software environment	37
5.3	Technology Stack	38
5.4	Application Interfaces	40
5.5	Key Code Excerpts	41
5.5.1	Webhook signature validation	41
5.5.2	Weighted health-score computation	42
5.5.3	BullMQ worker with bounded concurrency and retries	42
5.5.4	Resilient MongoDB connection	43
5.5.5	Real-time data fetching on the frontend	43
5.6	Automated Testing	44
5.6.1	Test strategy and pyramid	45
5.6.2	What is intentionally not covered	45
5.7	Containerization	46
5.8	Kubernetes Deployment with Helm	47
5.9	Jenkins CI/CD Pipeline	48
5.10	Engineering Challenges and Solutions	49
5.11	Security	51
5.12	Results	51
5.12.1	Performance metrics	52
5.12.2	End-to-end validation	52
5.12.3	Validated features	52
5.13	Conclusion	53
6	Critical Discussion, Limitations and Mitigations	54
6.1	Introduction	54
6.2	Methodological Limitations	54
6.2.1	AI classification accuracy measured on a limited sample	54
6.2.2	No formal usability study	54
6.3	Architectural Limitations	54
6.3.1	Single replica, no high availability	55
6.3.2	Single MongoDB instance, no replica set or backup policy	55
6.3.3	Secrets management without a vault	55
6.4	Data Privacy and Third-Party LLM Dependency	55
6.5	Operational and Process Limitations	56
6.6	Anticipated Jury Questions	56
6.7	Conclusion	57
7	Project Management	59
7.1	Introduction	59

7.2	Work Breakdown Structure	59
7.3	Risk Register	59
7.4	Effort Distribution	60
7.5	Conclusion	61
	General Conclusion and Future Work	62
	Bibliography	63
	Annexes	63
A	Full Helm Values File	64
B	Full Jenkinsfile	66
C	Additional Code Listings	68
C.1	Health-score service: full grade mapping	68
C.2	Knowledge-base service: signature generation	68
C.3	Frontend: Axios client with automatic token refresh	68
D	Glossary	70

List of Figures

1.1	Simplified organizational positioning of the internship	3
1.2	Project Gantt chart (academic year 2025–2026)	8
3.1	Activity diagram of the pipeline-analysis worker, with its two conditional branches	18
3.2	Global use-case diagram with generalization, include and extend relations	19
4.1	Overall architecture of the AIOps platform	25
4.2	UML component diagram with provided (solid) and required (dashed) dependencies	26
4.3	Physical deployment diagram of the lab environment	26
4.4	Sequence diagram: pipeline analysis	28
4.5	Full UML class diagram of the persistence layer (7 Mongoose collections + 4 embedded value objects)	30
4.6	Service-layer class diagram and its dependencies on the persistence layer	31

4.7	Sequence diagram: automatic merge-request risk comment	35
4.8	Sequence diagram: incident resolution and AI post-mortem generation .	35
5.1	Interface — Main dashboard	40
5.2	Interface — AI analysis detail	41
5.3	Interface — Security panel (vulnerabilities)	41
5.4	Test pyramid: implemented coverage vs. planned extensions	45
5.5	Kubernetes (k3s) deployment architecture	47
5.6	End-to-end CI/CD flow	49
7.1	Work breakdown structure (WBS)	59

List of Tables

1.1	Stakeholder analysis	4
1.2	SWOT analysis	5
1.3	Manual diagnosis time breakdown (average of 5 observed incidents) . .	7
1.4	Project objectives	8
2.1	Comparative synthesis of existing approaches	12
3.1	Non-functional requirements with measurable acceptance criteria	17
3.2	Use-case description: “Analyze a failed pipeline”	20
3.3	Use-case description: “Resolve an AI analysis”	21
3.4	Use-case description: “Manage an incident and generate a post-mortem”	22
3.5	Use-case description: “Comment a merge request with risk assessment”	23
3.6	Traceability matrix: objectives → features → use cases	24
4.1	Pipeline analysis processing steps	28
4.2	Main database collections	29
4.3	Health-score metrics and weights	32
4.4	Score-to-grade mapping	32
4.5	Main REST API endpoints	33
5.1	Backend stack	38
5.2	Frontend stack	39
5.3	Infrastructure and DevOps stack	40
5.4	Automated test suite	44

5.5	Coverage gaps and manual mitigation	46
5.6	Selected infrastructure issues encountered and their resolution	50
5.7	Security measures	51
5.8	Performance results	52
6.1	Anticipated jury questions and prepared answers	57
7.1	Risk register	60
7.2	Approximate effort distribution	61

List of Abbreviations

AIOps	Artificial Intelligence for IT Operations
CI/CD	Continuous Integration / Continuous Deployment
LLM	Large Language Model
RBAC	Role-Based Access Control
JWT	JSON Web Token
HMAC	Hash-based Message Authentication Code
MTTR	Mean Time To Resolve
CVE	Common Vulnerabilities and Exposures
MR	Merge Request
API	Application Programming Interface
ODM	Object Document Mapper
SPA	Single Page Application
PVC	Persistent Volume Claim
k3s	Lightweight Kubernetes distribution
KB	Knowledge Base

General Introduction

Modern software engineering relies heavily on DevOps methodologies, which bring development and operations teams closer together to deliver faster and more reliably. At the heart of this approach, Continuous Integration and Continuous Deployment (CI/CD) pipelines automate the building, testing and release of software. This automation, however, has a downside: it produces a considerable amount of technical data — execution logs, code-quality reports, vulnerability scans — whose manual exploitation quickly becomes unmanageable.

When a pipeline fails, the engineer must scan thousands of log lines, cross-reference the results of several heterogeneous tools and identify the root cause alone. This diagnosis takes, on average, 30 to 90 minutes per incident, often for recurring problems that were already solved in the past but never capitalized.

This is the context of the present final year project, which proposes an **AIOps platform** applying artificial intelligence to the supervision of CI/CD pipelines. The goal is to automate analysis, centralize data sources, provide AI-assisted root-cause diagnosis, and capitalize recurring solutions.

This report is organized into seven chapters. The **first chapter** presents the general context, the host organization, the study of the existing situation and the proposed solution. The **second chapter** surveys the state of the art and positions the project's contribution relative to existing commercial and academic work. The **third chapter** covers the analysis and specification of functional and non-functional requirements. The **fourth chapter** details the design of the solution, its architecture, data models, services and APIs. The **fifth chapter** describes the implementation, testing, deployment (Docker, Helm, Kubernetes, Jenkins) and the results obtained. The **sixth chapter** critically discusses the limitations of the solution and the mitigations identified. Finally, the **seventh chapter** covers the project-management aspects (work breakdown, risk register), before the general conclusion, future work and annexes.

Chapter 1

General Context and Problem Statement

1.1 Introduction

This chapter sets the framework of the project. It introduces the host organization, analyzes the current situation and its limitations, and presents the proposed solution as well as the working methodology.

1.2 Host Organization

This project was carried out at **Capgemini — Altran Telnet Corporation (ATC) Tunisia**. ATC is a joint venture created in 2008 between the Altran group and the Tunisian group Telnet Holding. The company specializes in engineering and the development of embedded software and real-time systems, notably for the defense, security, aeronautics and automotive sectors.

Following the acquisition of Altran by the Capgemini group in 2020, the entity progressively joined the **Capgemini Engineering** brand, the business line dedicated to engineering and R&D services within the Capgemini group. Capgemini is today a global leader in consulting, technology services and digital transformation, present in more than 50 countries. Locally, ATC supports several international client companies in their DevOps and platform-engineering transformation, where engineering teams face failing CI/CD pipelines on a daily basis — precisely the operational pain point addressed by this project.

1.2.1 Organizational positioning

Figure 1.1 situates the internship within the company’s engineering organization. The project was hosted by a DevOps/Platform Engineering unit, itself part of a software engineering department reporting to the Capgemini Engineering Tunisia management. This unit is responsible, among other duties, for maintaining the CI/CD tooling (GitLab, Jenkins, SonarQube) used by several client-facing engineering teams — which made it

the natural sponsor for an AIOps initiative aimed at those very pipelines.

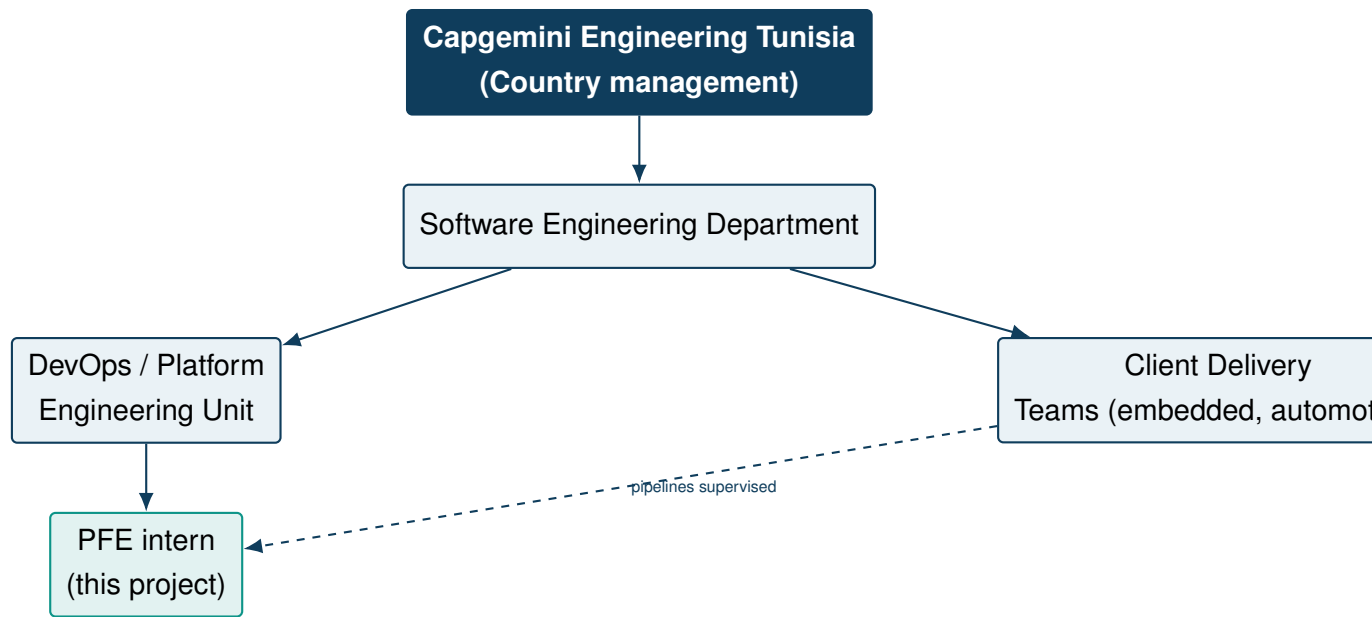


Figure 1.1: Simplified organizational positioning of the internship

1.2.2 Stakeholder analysis

Table 1.1 identifies the stakeholders of the project and their respective interest and influence, which guided prioritization decisions throughout the sprints (e.g. dashboard and root-cause analysis were prioritized over the weekly report because they directly serve the highest-influence stakeholder).

Table 1.1: Stakeholder analysis

Stakeholder	Interest	Influence	Expectation
DevOps unit manager	High	High	Reduce time engineers spend triaging pipeline failures
Client delivery engineers	High	Medium	Faster, clearer diagnosis without learning a new tool
Academic supervisor (ESPRIT)	Medium	High	Rigorous methodology, measurable results, sound architecture
Security/Compliance officer	Medium	Medium	No leakage of secrets to third-party LLM APIs
PFE intern (author)	High	Medium	Deliver a production-grade, defensible final-year project

1.2.3 SWOT analysis of the initiative

Table 1.2: SWOT analysis

Strengths	End-to-end automation (ingestion to remediation suggestion); self-feeding knowledge base reduces recurring LLM cost over time; fully containerized and reproducible on Kubernetes
Weaknesses	Dependency on a third-party LLM provider for the core analysis; single-replica deployment in the current lab environment; limited automated test coverage beyond unit tests
Opportunities	Extensible to other CI/CD platforms (GitHub Actions, Azure DevOps); potential integration with existing observability stacks (Prometheus/Grafana) already used by the company
Threats	LLM API pricing or rate-limit changes; data-privacy constraints on sending log excerpts to an external API for client projects with strict compliance requirements

These weaknesses and threats are revisited in detail, with concrete mitigations, in the dedicated Critical Discussion chapter at the end of this report.

1.3 Study of the Existing Situation

In current practice, diagnosing a failed pipeline relies on an entirely manual process. The engineer must:

- scan thousands of CI log lines manually;
- cross-reference the results of several independent tools (GitLab CI, SonarQube, Trivy) in separate interfaces;
- identify the root cause without any automated assistance;
- manage incidents without centralized traceability;
- waste time on recurring problems whose solutions are not documented.

Furthermore, several tools exist on the market but remain siloed: GitLab CI provides logs, SonarQube analyzes code quality, Trivy detects image vulnerabilities, while classic observability stacks (Prometheus, Grafana, ELK) focus on infrastructure metrics rather

than pipeline-level root-cause diagnosis. Commercial APM/AIOps platforms (Datadog, Dynatrace) offer advanced capabilities but remain costly, generic, and not tailored to CI/CD-specific failure analysis.

1.4 Critique of the Existing Situation

The analysis highlights the following limitations:

- **Scattered data** that is never correlated across sources;
- **No automated, contextualized root-cause analysis**;
- **No memorization** of solutions to recurring problems;
- **Alerting without diagnosis**, which reports failures without explaining their origin;
- **High diagnosis time** (30 to 90 minutes) and poor incident traceability.

To make this critique measurable rather than anecdotal, an informal time-tracking exercise was conducted during the first sprint: five recent pipeline failures handled manually by the host team were timed end-to-end and decomposed into the phases of Table 1.3. The decomposition shows that more than half of the diagnosis time is spent merely *locating* the relevant information across tools, before any actual reasoning about the root cause begins — precisely the phase an AIOps platform can compress to seconds.

Table 1.3: Manual diagnosis time breakdown (average of 5 observed incidents)

Phase	Avg. time	Observation
Locating the failing job and reading raw logs	12–20 min	Logs are paginated and not searchable across jobs
Cross-checking SonarQube / Trivy dashboards	8–15 min	Separate login, separate UI, no link from the CI log
Formulating and testing a hypothesis	8–40 min	Highly variable; repeated for already-seen errors
Communicating the finding (Slack, ticket)	2–15 min	No persisted, searchable record for next time
Total	30–90 min	Matches the range stated in objective O1

1.5 Proposed Solution

To address these limitations, we propose an **AIOps platform** that automates CI/CD pipeline analysis, centralizes GitLab CI, SonarQube and Trivy results in a single interface, memorizes solutions in a self-feeding knowledge base, continuously measures project health, and manages incidents with full traceability and AI-generated post-mortems. Table 1.4 summarizes the project objectives.

Table 1.4: Project objectives

ID	Description
O1	Reduce pipeline diagnosis time from 30–90 min to under 2 min using AI
O2	Centralize GitLab CI, SonarQube and Trivy data in a single interface
O3	Provide AI root-cause analysis with severity-prioritized fix suggestions
O4	Detect and alert on container security vulnerabilities (Trivy CVEs)
O5	Learn recurring solutions in a knowledge base, avoiding repeated LLM calls
O6	Offer a per-project health score for continuous quality steering
O7	Manage the incident lifecycle with AI-generated post-mortems
O8	Assess Merge Request risk before merging, with an automatic comment
O9	Produce weekly aggregated DevOps metrics reports

1.6 Working Methodology

The project was conducted using an agile approach inspired by **Scrum**, organized into eight iterative sprints of roughly three weeks each, spread over the 2025–2026 academic year. Each sprint delivered a functional increment (webhook ingestion, AI analysis, dashboard, health score, incidents, knowledge base, containerization, CI/CD), validated before moving on to the next. Figure 1.2 presents the resulting project planning.

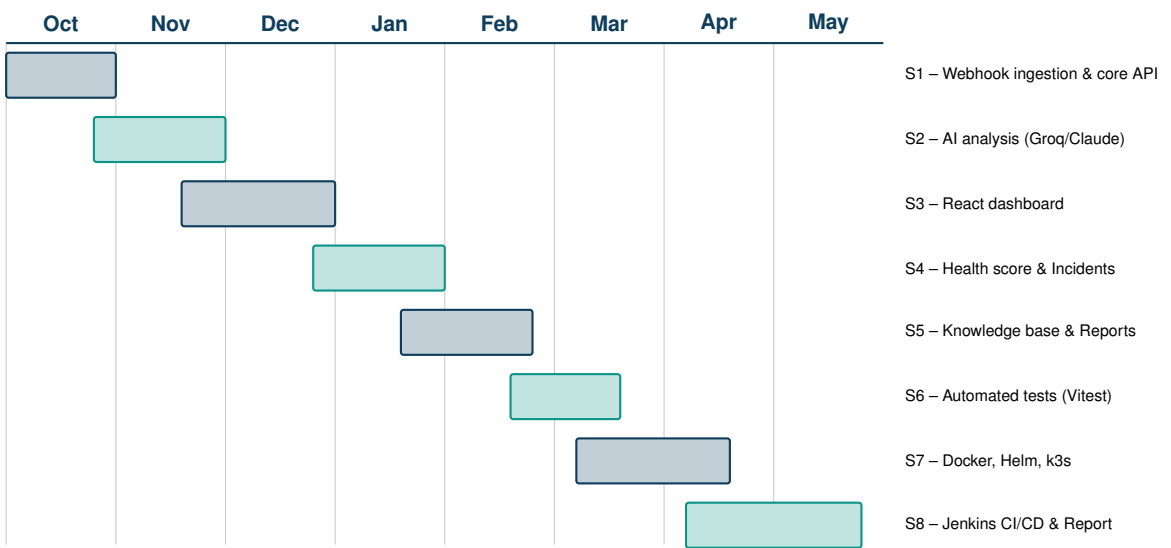


Figure 1.2: Project Gantt chart (academic year 2025–2026)

1.7 Conclusion

This chapter introduced the project context, the host organization and the limitations that justify the proposed solution. Before specifying the requirements, the next chapter surveys how existing commercial and academic work addresses — partially — the same problem, in order to position precisely the contribution of this project.

Chapter 2

State of the Art and Comparative Study

2.1 Introduction

AIOps is an active commercial and research field. This chapter surveys the main categories of existing solutions — commercial AIOps/APM suites, CI/CD-native tooling, and academic work on log-based root-cause analysis — and positions the contribution of this project relative to each.

2.2 Commercial AIOps and Observability Platforms

2.2.1 Datadog and Dynatrace

Datadog and Dynatrace are the two dominant commercial Application Performance Monitoring (APM) suites with AIOps modules (Datadog Watchdog, Dynatrace Davis AI). Both excel at infrastructure and application-level anomaly detection (latency spikes, error-rate deviations) using statistical and ML models trained on metrics and traces. However, neither is designed around *CI/CD pipeline* semantics: they do not natively parse GitLab/GitHub CI job logs, do not cross-reference a SonarQube quality gate with a Trivy CVE scan for the same commit, and their root-cause narratives are generated from telemetry correlation rather than from a generative-AI reading of the actual failure context. They are also priced per host/container ingested, which makes them costly for a use case centered on CI pipelines rather than production infrastructure.

2.2.2 Moogsoft and BigPanda

Moogsoft and BigPanda focus on *alert correlation*: deduplicating and clustering thousands of monitoring alerts into a smaller number of actionable incidents. This is conceptually close to the correlation service implemented in this project, but again operates at the infrastructure-alert level, not at the CI/CD pipeline-failure level, and does not produce a natural-language root-cause explanation or a concrete code-level fix

suggestion.

2.2.3 PagerDuty / Jeli and incident-management tools

PagerDuty (and its acquisition Jeli) provide mature incident lifecycle management with timelines, on-call escalation and post-incident reviews — directly comparable to the Incident model and post-mortem feature of this project. Their post-mortems, however, remain manually authored templates; they do not auto-generate a draft post-mortem from an LLM reading the incident timeline and the associated technical root cause, which is what this project automates.

2.3 CI/CD-Native Tooling

GitLab CI, SonarQube and Trivy — the three data sources integrated by this platform — each solve one slice of the problem in isolation: GitLab CI executes and logs the pipeline, SonarQube evaluates static code quality, and Trivy scans container images for known CVEs. None of the three correlates its own output with the other two, and none performs root-cause reasoning over a failure; they report *what* happened, never *why*, nor what to do about it. GitHub’s own “Copilot for pipelines” / GitHub Actions failure summaries (announced in 2024) point in the same direction as this project but remain tied to the GitHub ecosystem and, at the time of writing, only summarize a single job log without cross-tool correlation or a persisted, searchable knowledge base.

2.4 Academic Background: LLMs for Log Analysis and RCA

Recent academic work (e.g. studies on LogGPT-style log-anomaly detection, and on using LLMs for incident root-cause summarization in large-scale cloud operations [1]) confirms two findings directly relevant to this project’s design: (1) general-purpose LLMs, when given well-structured context (relevant log excerpts rather than entire raw logs), perform competitively with specialized log-parsing models on root-cause classification, which justifies the normalization/correlation step before the LLM call rather than feeding raw logs directly; and (2) caching/memorizing previously diagnosed patterns substantially reduces both cost and latency for recurring failures — the exact rationale behind this project’s knowledge-base service.

2.5 Comparative Synthesis

Table 2.1 summarizes the comparison along the dimensions most relevant to the project’s objectives (Table 1.4).

Table 2.1: Comparative synthesis of existing approaches

Solution	CI/CD-aware	Multi-source correlation	AI root cause (generative)	Self-feeding KB	Cost profile
Datadog / Dynatrace	No	Partial (infra only)	No (statistical)	No	High (per host)
Moogsoft / BigPanda	No	Yes (alerts)	No	No	High
PagerDuty / Jeli	Partial	No	No	No (manual)	Medium
GitHub Copilot for CI	Yes	No	Partial (single source)	No	Included in plan
This project	Yes	Yes (Git-Lab+Sonar+Trivy)	Yes (LLM)	Yes	Low (LLM API + cache)

2.6 Positioning of the Contribution

This project does not claim to out-perform mature commercial APM suites on infrastructure-level anomaly detection, which is not its goal. Its contribution is narrower and more specific: a CI/CD-native, multi-source-correlated, generative-AI-assisted root-cause platform with an organizational-memory mechanism (the knowledge base), delivered at a fraction of the licensing cost of commercial AIOps suites because it relies on pay-per-call LLM APIs rather than per-host monitoring agents. This positioning directly motivated the architectural choices detailed in Chapter 4.

2.7 Conclusion

This survey confirms that no single existing tool covers the specific intersection of capabilities targeted by objectives O1–O9: CI/CD-native ingestion, multi-source correlation, generative root-cause analysis, and a self-feeding knowledge base. The next chapter formalizes the requirements that operationalize this positioning.

Chapter 3

Requirements Analysis and Specification

3.1 Introduction

This chapter identifies the platform actors, lists the functional and non-functional requirements, and formalizes them through use-case modeling.

3.2 Actor Identification

The platform relies on Role-Based Access Control (RBAC) with three levels:

- **Administrator:** all operations (deletion, recomputation, configuration);
- **Analyst:** analyses, resolution, comments and post-mortems;
- **Viewer:** read-only access.

A **system actor** is also involved: the GitLab server, which emits pipeline and merge-request events through webhooks.

3.3 Functional Requirements

3.3.1 Dashboard

The dashboard is the system's landing page and must give an engineer, within seconds of logging in, a synthetic view of the state of all supervised projects: the list of recent pipelines with their status, the global success rate over the last seven days plotted as a trend chart, a small set of key performance indicators (active incidents, average MTTR, open critical vulnerabilities), a ranked list of the top recurring issues across projects, and a live feed of notifications pushed over WebSocket as new analyses complete. The dashboard must remain usable with a global project filter, since an engineer typically cares about a subset of the supervised projects rather than the entire fleet.

3.3.2 Pipeline management

The system must expose a paginated, filterable list of pipelines (filterable by status, branch, project and date range), since the number of pipelines grows continuously and an unbounded list would not scale. Selecting a pipeline must reveal its full detail: the list of jobs and their individual status, the raw logs of the jobs that failed, and — when available — the associated AI analysis. An analyst must also be able to manually re-trigger the analysis of a given pipeline, for instance after fixing the underlying tool configuration and wanting to confirm the diagnosis was accurate.

3.3.3 AI analysis — Root-Cause Analysis

For every analyzed pipeline, the system must classify the failure into one of a fixed set of error types (build failure, test failure, dependency issue, security vulnerability, configuration error, or unknown), produce a natural-language explanation of the root cause, assign a risk level (critical/high/medium/low) together with a numeric confidence score, and propose one or more severity-prioritized fix suggestions, each optionally carrying a ready-to-run shell command or a code hint. The system must also list the files it identified as affected, and must detect when a failure matches a previously seen pattern so the recurring-issue view (Section 3.3.1, Dashboard) can surface it.

3.3.4 Security and code quality

The system must integrate Trivy’s vulnerability scan results, exposing severity counters (critical/high/medium/low), the full list of detected CVEs with package and fixed-version information, and a workflow to mark a given vulnerability as acknowledged or ignored. It must equally integrate SonarQube metrics — quality-gate status, bug count, vulnerability count, code-smell count and test coverage — since these feed directly into the health-score computation (Section 4.6.1) and into the AI analysis context.

3.3.5 Knowledge base

Whenever an analyst resolves an analysis, the system must automatically memorize the validated solution under a unique signature derived from the error type and root cause, so that a future occurrence of the same error is answered instantly from the cache rather than triggering a new, costlier LLM call. The knowledge base must also be independently browsable and searchable (full-text, across title, tags, error type and root cause) so an analyst can proactively consult it before even waiting for an automated analysis.

3.3.6 Health score, incidents and reports

The system must continuously compute, per project, a composite health score between 0 and 100, mapped to a letter grade from A to F, and must expose its full breakdown (the six weighted metrics of Table 4.3) rather than the score alone, so that an engineer understands *why* a project is graded the way it is. Failed pipelines must automatically open an incident, which the system tracks through a lifecycle (open → investigating → resolved) with a timestamped timeline, an MTTR computation, and an on-demand AI-generated post-mortem once resolved. Open merge requests must receive an automatic risk comment summarizing the latest analysis. Finally, the system must produce a weekly aggregated report per project, comparable week-over-week, summarizing pipeline volume, success rate, new vulnerabilities and incident count.

3.4 Non-Functional Requirements

- **Security:** JWT authentication, three-level RBAC, HMAC-SHA256 webhook validation, Bcrypt hashing, HTTP security headers.
- **Performance:** pipeline analysis in under ten seconds.
- **Real time:** dashboard updates over WebSocket.
- **Scalability:** asynchronous processing via message queues and concurrent workers.
- **Reliability:** error recovery with retries and exponential backoff.
- **Maintainability:** layered architecture, modular and documented code.
- **Portability:** full containerization and reproducible Kubernetes deployment.

Each qualitative requirement above is restated in Table 3.1 as a measurable acceptance criterion, together with the verification method actually used during this project (test, measurement, or code inspection), so that the results reported in Chapter 5 can be checked against an explicit target rather than a vague intention.

Table 3.1: Non-functional requirements with measurable acceptance criteria

NFR	Measurable acceptance criterion	Verification method
Security	100% of webhook requests with an invalid HMAC signature rejected with HTTP 401; passwords never stored in plaintext	Manual penetration test + code review
Performance	End-to-end analysis (webhook received → result persisted) <10 s on 95% of observed pipelines	Timestamp logging across 20+ real runs
Real time	Dashboard reflects a new analysis within 500 ms of completion	Browser network-tab measurement
Scalability	Backend remains stateless; horizontal scale-out validated by running 2 replicas of the worker concurrently without job duplication	Manual load test with <code>kubect1 scale</code>
Reliability	A transient GitLab/SonarQube/Trivy API failure is retried up to 3 times before the job is marked failed	Fault injection (forced 500 responses)
Maintainability	Each service module exposes ≤ 10 public functions and has a single responsibility	Code review against the architecture in Figure 4.6
Portability	The full stack starts from a clean k3s cluster with a single <code>helm install</code> command	Manual redeployment from scratch

3.5 Functional Behavior: Activity Diagram

Beyond the step list of Table 4.1 (detailed in Chapter 4), Figure 3.1 models the decision logic of the pipeline-analysis worker as a UML activity diagram, making explicit the two conditional branches that are easy to overlook in a purely sequential reading: the knowledge-base cache hit/miss branch, and the pipeline success/failure branch that determines whether an incident is created.

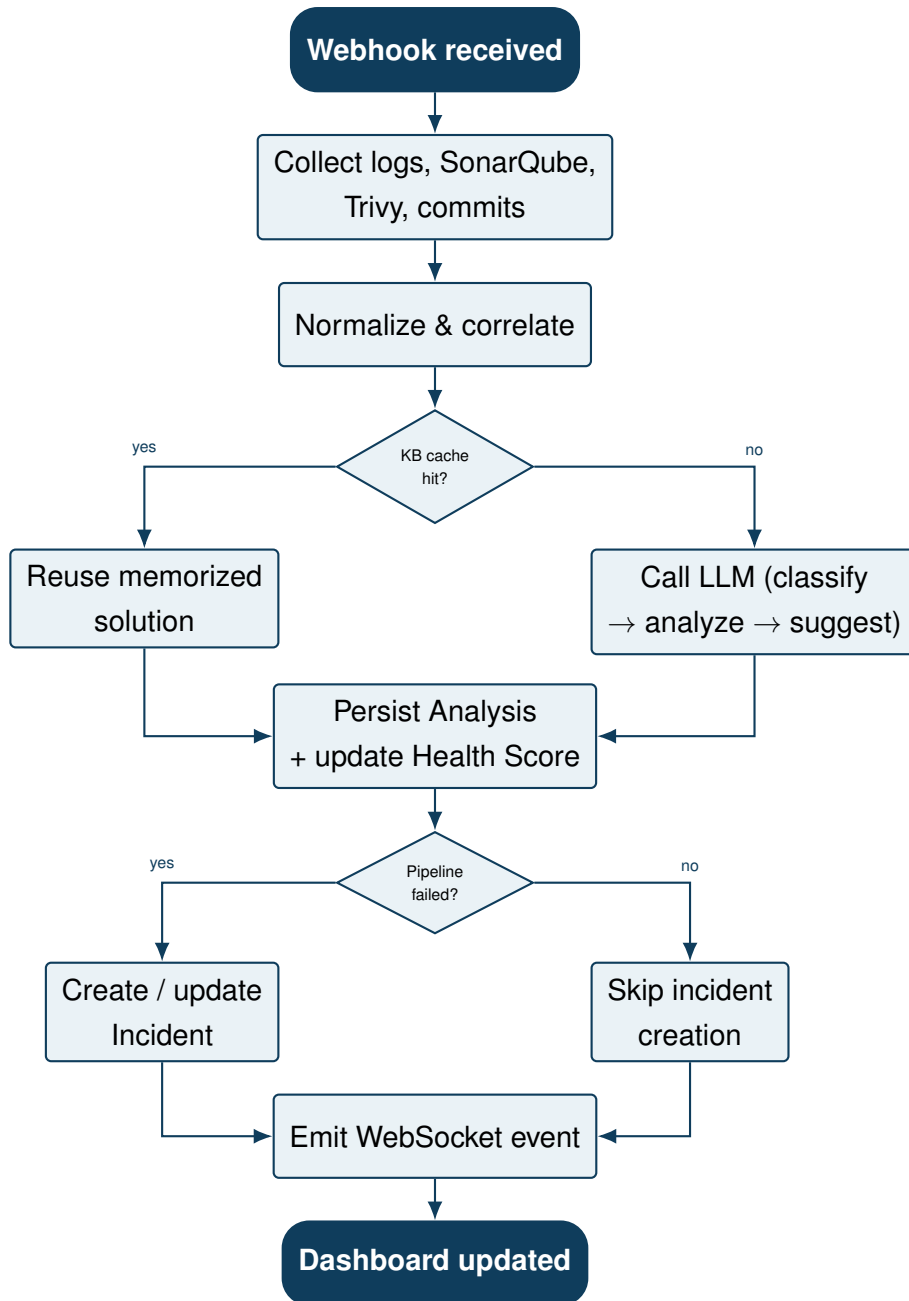


Figure 3.1: Activity diagram of the pipeline-analysis worker, with its two conditional branches

3.6 Use-Case Modeling

Figure 3.2 presents the global use-case diagram, showing the three human roles and the GitLab system actor interacting with the main capabilities of the platform. Generalization arrows (hollow triangle) express that *Analyst* and *Viewer* both inherit the read-only capabilities of a base *Authenticated User*, while *Administrator* additionally inherits every capability of *Analyst*. Dashed arrows annotated «include» denote a mandatory sub-behavior systematically triggered by the base use case, and «extend»

denotes an optional behavior triggered only under a condition (e.g. a cache miss).

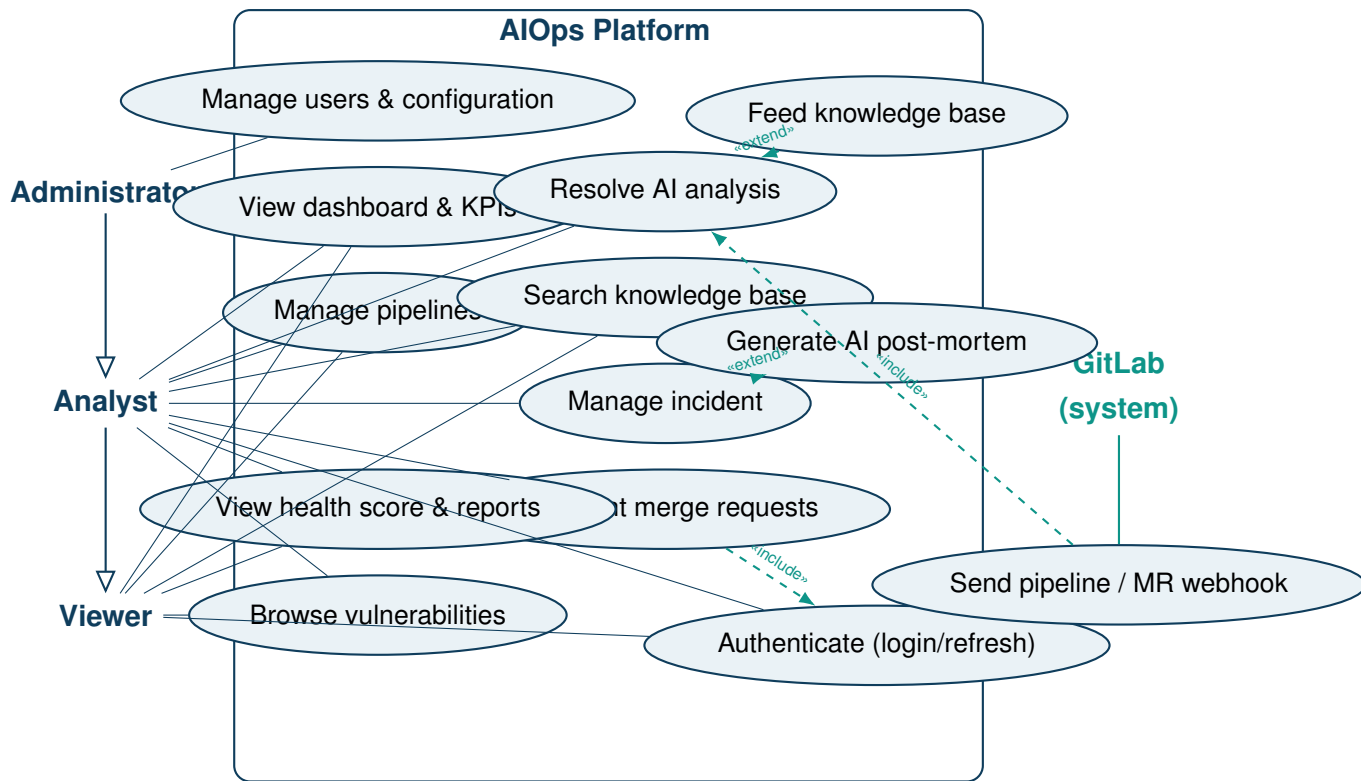


Figure 3.2: Global use-case diagram with generalization, include and extend relations

3.6.1 Use-case refinement

Tables 3.2, 3.3, 3.4 and 3.5 textually describe the four most critical use cases of the system.

Table 3.2: Use-case description: “Analyze a failed pipeline”

Element	Description
Primary actor	GitLab server (system), Analyst
Secondary actors	Groq/Claude LLM, SonarQube, Trivy
Precondition	A pipeline has finished on a supervised project; the webhook secret is configured
Main scenario	1. GitLab emits a signed webhook. 2. The system validates the HMAC signature. 3. An analysis job is enqueued (202 Accepted). 4. The worker collects logs, SonarQube issues, Trivy CVEs and commits. 5. The system normalizes and correlates the data. 6. « <i>include</i> » the knowledge base is queried by signature. 7. On a cache miss, the LLM classifies the error and produces the root cause and fix suggestions. 8. The result is persisted and broadcast in real time.
Postcondition	An analysis is available; an incident is auto-created if the pipeline failed
Alternate flow	Cache hit at step 6: the memorized solution is reused, steps skipped, LLM not called
Exceptions	Invalid HMAC signature (request rejected, 401); GitLab/SonarQube/Trivy API timeout (retried 3 times with exponential backoff)

Table 3.3: Use-case description: “Resolve an AI analysis”

Element	Description
Primary actor	Analyst
Precondition	The analyst is authenticated with at least the <i>Analyst</i> role; an unresolved analysis exists
Main scenario	1. The analyst opens the pipeline detail page. 2. The analyst reviews the AI root cause and applies the suggested fix. 3. The analyst marks the analysis as resolved (PATCH <code>/api/analyses/:id/resolve</code>). 4. « <i>extend</i> » the system feeds the knowledge base with the validated solution, keyed by the error signature. 5. The MTTR is computed and stored.
Postcondition	The analysis is flagged <code>resolved=true</code> ; the knowledge base contains a new or updated entry
Exceptions	Role insufficient (403 Forbidden); analysis already resolved (idempotent no-op)

Table 3.4: Use-case description: “Manage an incident and generate a post-mortem”

Element	Description
Primary actor	Analyst
Precondition	An incident exists (auto-created from a failed pipeline analysis)
Main scenario	1. The analyst opens the incident detail (timeline, linked analysis). 2. The analyst updates the status (open → investigating → resolved), each change appended to the timeline. 3. The analyst may add free-text comments to the timeline. 4. Once resolved, the analyst triggers post-mortem generation. 5. «<i>extend</i>» the LLM summarizes the timeline and analysis into a structured post-mortem with recommendations.
Postcondition	The incident timeline is complete; a post-mortem document is attached to the incident
Exceptions	Post-mortem generation requested before resolution (rejected); LLM call failure (retried, then degraded to a template-based summary)

Table 3.5: Use-case description: “Comment a merge request with risk assessment”

Element	Description
Primary actor	GitLab server (system)
Secondary actor	Groq/Claude LLM
Precondition	A merge-request webhook (open or update action) is received
Main scenario	1. The system identifies the most recent analysis for the target project. 2. The MR-comment service computes a 0–100 risk score from the analysis severity and confidence. 3. <i>«include»</i> the system authenticates against the GitLab API using a stored token. 4. The LLM generates a three-sentence French explanation of the risk. 5. The system posts a formatted Markdown comment on the merge request.
Postcondition	The merge request displays an automatic risk comment
Exceptions	No analysis available yet for the project (comment skipped); GitLab API rate limit reached (comment deferred and retried)

3.7 Requirements Traceability Matrix

Table 3.6 maps each functional objective defined in Chapter 1 to the concrete features, REST endpoints and use cases that satisfy it, ensuring no requirement was left unimplemented.

Table 3.6: Traceability matrix: objectives $\rightarrow$ features $\rightarrow$ use cases

Obj.	Feature(s) implemented	Use case
O1	AI root-cause analyzer, BullMQ async pipeline	Table 3.2
O2	GitLab/SonarQube/Trivy services + normalizer + correlation service	Table 3.2
O3	<code>ai/rootCause.analyzer.js</code> , <code>fixSuggester.js</code>	Table 3.2
O4	Trivy service, Vulnerability model, security panel	Figure 4.6
O5	Knowledge-base service, signature cache	Table 3.3
O6	Health-score service (6 weighted metrics, grade A–F)	Section 4.6.1
O7	Incident model, timeline, AI post-mortem	Table 3.4
O8	MR-comment service, risk scoring	Table 3.5
O9	Weekly-report service, week-over-week comparison	Section 4.6.3

3.8 Conclusion

This chapter formalized the platform requirements through actor identification, functional and non-functional specification, use-case modeling and a traceability matrix. The next chapter translates them into a detailed technical design.

Chapter 4

Design and Architecture

4.1 Introduction

This chapter presents the overall architecture, the processing flow, the data models, the business services and the API and frontend design.

4.2 Overall Architecture

The platform follows an event-driven architecture. GitLab emits a webhook at the end of each pipeline (or on a merge-request event); the backend validates the request, enqueues it in a message queue, then asynchronous workers perform collection, normalization, correlation and AI analysis. Results are persisted in MongoDB and pushed to the React dashboard in real time over WebSocket. Figure 4.1 gives the global view of the architecture.

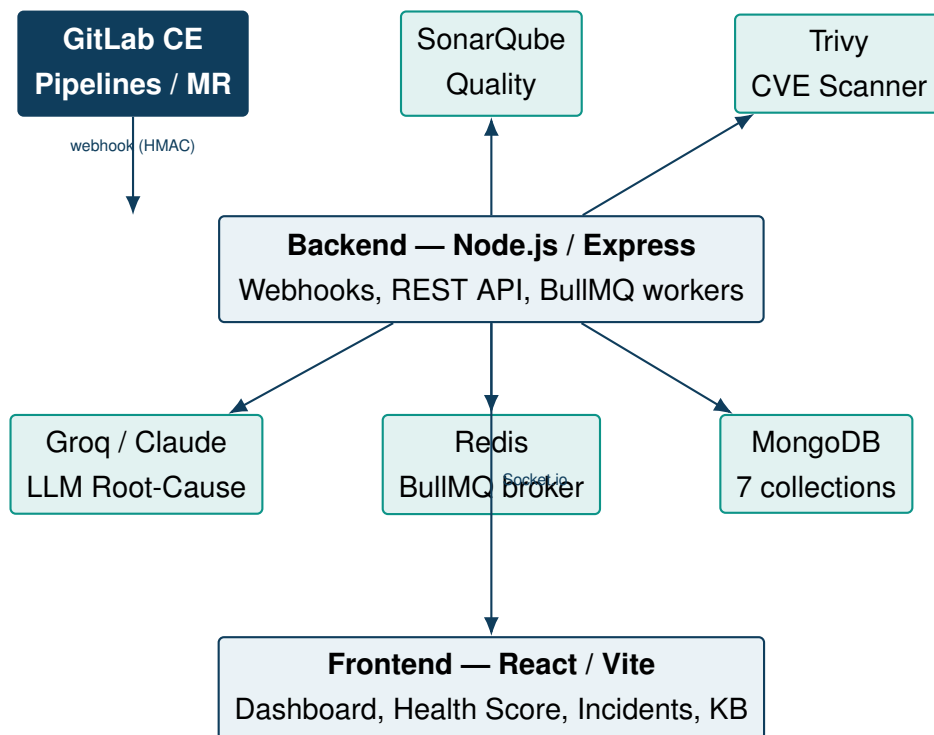


Figure 4.1: Overall architecture of the AIOps platform

4.2.1 Component diagram

Figure 4.2 refines the architecture into UML components with explicit provided/required interfaces (lollipop/socket notation), showing how the backend exposes a REST API and a webhook endpoint while requiring the three external CI/CD data sources and the LLM provider.

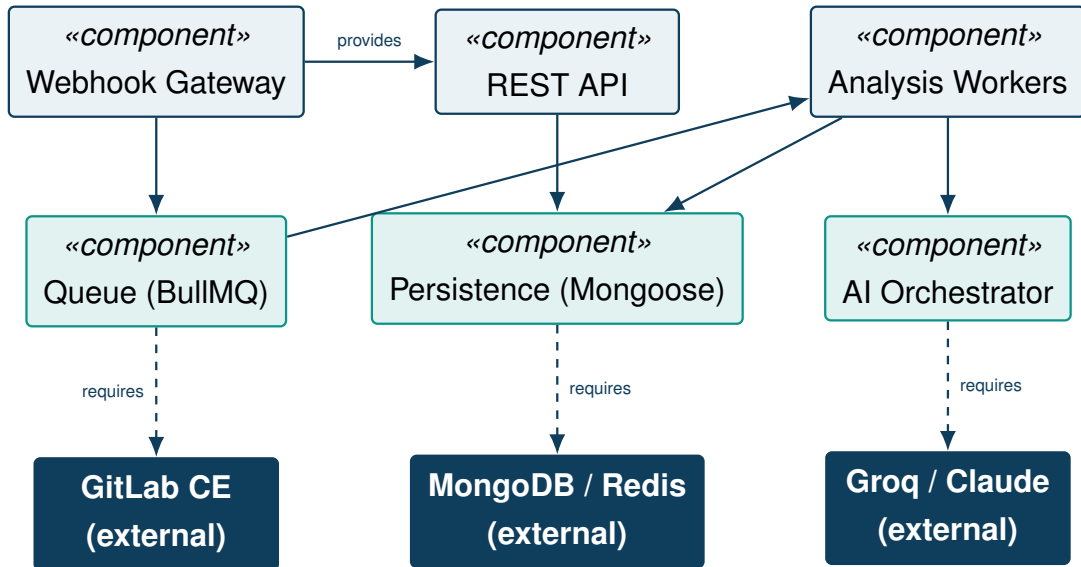


Figure 4.2: UML component diagram with provided (solid) and required (dashed) dependencies

4.2.2 Physical deployment diagram

Whereas Figure 5.5 (Chapter 5) shows the *logical* topology inside the Kubernetes cluster, Figure 4.3 shows the *physical* deployment nodes used throughout the project, including the developer workstation and the external SaaS dependencies that the cluster relies on.

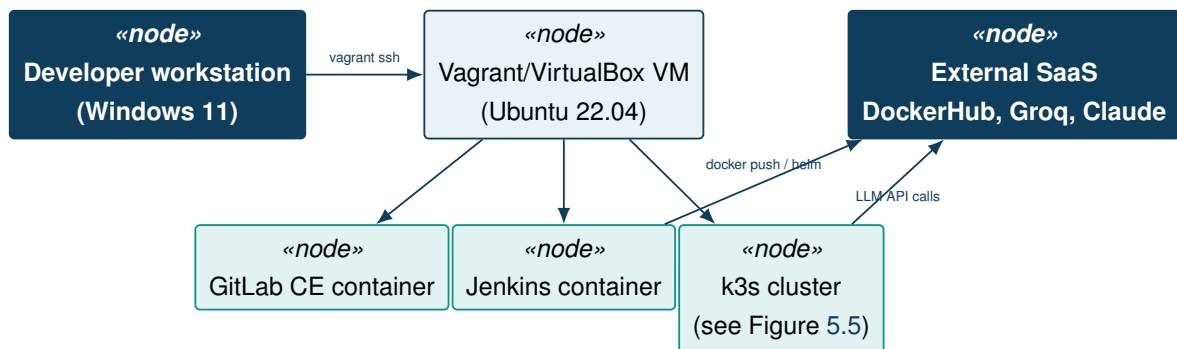


Figure 4.3: Physical deployment diagram of the lab environment

4.3 Backend Architecture

The backend is a Node.js (ESM) / Express application organized in layers, as outlined below.

```

1 backend/src/
2 |-- server.js           # HTTP entry point (port 3001)
3 |-- app.js             # Express app + global middlewares
4 |-- socket.js          # Socket.io initialization
5 |-- config/            # env, database (Mongoose), redis (ioredis
6   )
7 |-- api/               # controllers, middlewares, routes
8 |-- ai/               # groq/claude clients, analyzer, prompts
9 |-- models/           # 7 Mongoose schemas
10 |-- queues/           # BullMQ queues + workers
11 |-- services/         # gitlab, sonarqube, trivy, correlation,
    ...
11 \-- utils/           # logger (Winston), httpClient, retry

```

Listing 4.1: Backend top-level structure

The layers are: the *API* layer (controllers, middlewares, routes), the business *services* (GitLab, SonarQube, Trivy, normalization, correlation, health score, MR comment, weekly report), the *queues and workers* (BullMQ over Redis), the *AI* layer (LLM clients, root-cause orchestrator, prompts) and the *models* layer (Mongoose schemas).

4.4 Main Processing Flow

When a pipeline finishes, GitLab sends a webhook to `POST /api/webhooks/gitlab`. The HMAC middleware validates the signature, a rate limiter caps requests, and the controller enqueues a job in the `pipeline-analysis` BullMQ queue, returning `202 Accepted` immediately. A worker (5 concurrent) then executes the steps summarized in Table 4.1 and illustrated in Figure 4.4.

Table 4.1: Pipeline analysis processing steps

Step	Action
1	Fetch GitLab CI logs of failed jobs
2	Fetch SonarQube issues (bugs, vulnerabilities, code smells, quality gate)
3	Fetch Trivy CVEs from the CI artifact <code>trivy-results.json</code>
4	Fetch the last commits (author, message, diff)
5	Normalize all sources into a unified object
6	Knowledge-base lookup: cache hit reuses the known solution; cache miss calls the LLM (classify → analyze → suggest)
7	Persist the analysis, upsert the knowledge base, recompute the health score
7b	Auto-create an incident if the pipeline failed (one per pipeline)
7c	Post an automatic risk comment on the GitLab MR if open
8	Emit a WebSocket event to update the dashboard instantly

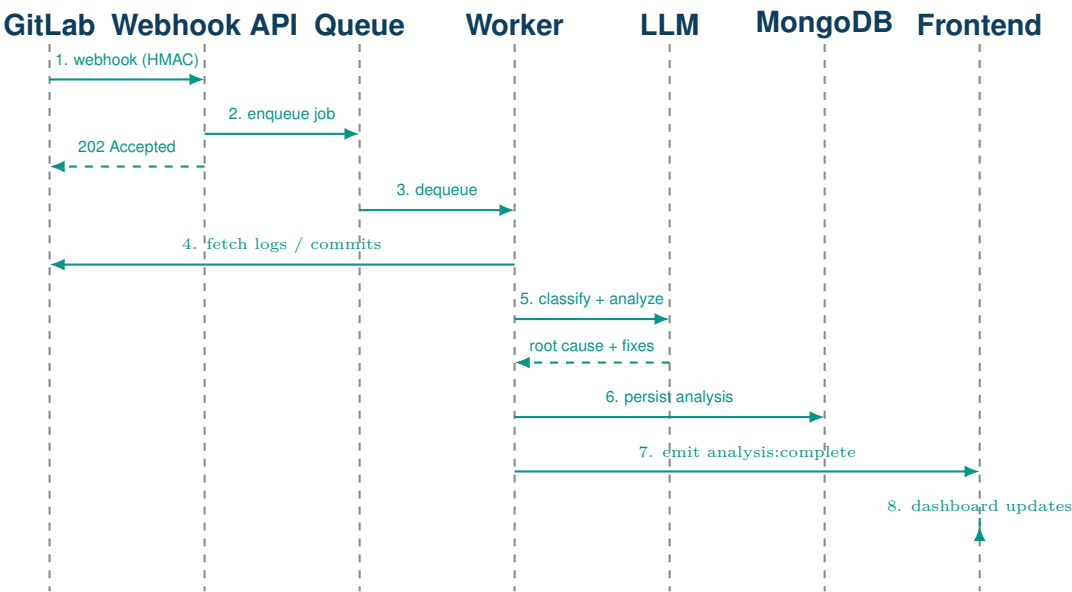


Figure 4.4: Sequence diagram: pipeline analysis

4.5 Data Models

The MongoDB database relies on seven main collections, described in Table 4.2 and Figure 4.5.

Table 4.2: Main database collections

Collection	Role
User	User accounts and roles (RBAC)
Pipeline	GitLab pipeline metadata and status
Analysis	AI analysis results (root cause, suggestions, confidence, risk)
Vulnerability	Trivy-detected vulnerabilities (CVEs)
KnowledgeBase	Memorized solutions and error signatures
ProjectHealth	Health score and per-project breakdown
Incident	Incidents, timeline and AI post-mortems

Figure 4.5 gives the full UML class diagram of the persistence layer: every attribute is typed, embedded sub-documents are attached to their owner through a composition relation (filled diamond, since they cannot exist independently of their parent document), and associations between top-level collections carry explicit multiplicities.

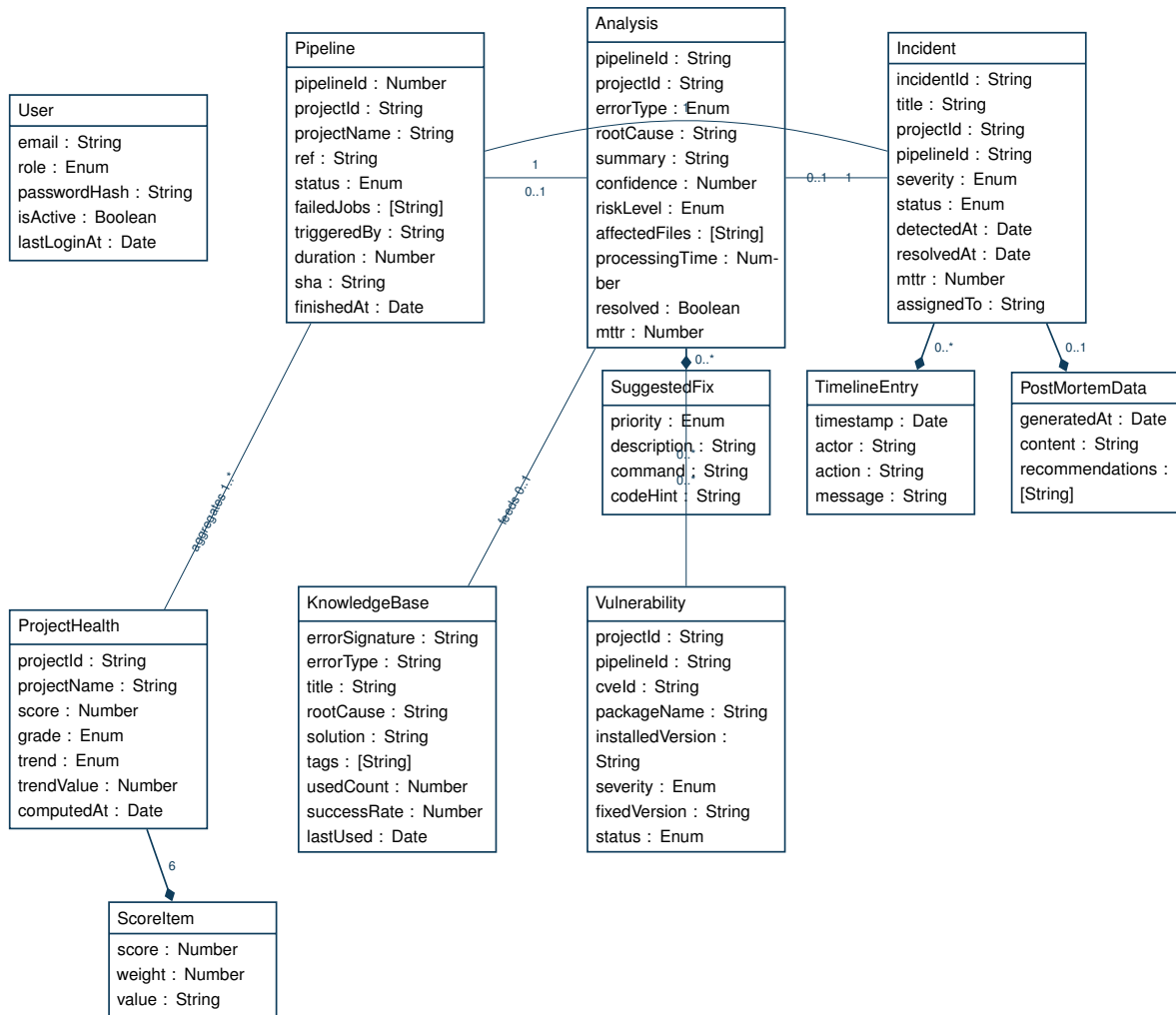


Figure 4.5: Full UML class diagram of the persistence layer (7 Mongoose collections + 4 embedded value objects)

The diagram highlights three modeling decisions. First, **SuggestedFix**, **TimelineEntry**, **PostMortemData** and **ScoreItem** are deliberately modeled as **embedded sub-documents** rather than separate collections: they have no independent identity or lifecycle outside their parent (a composition in UML terms), and MongoDB’s document model lets them be fetched atomically with their owner in a single query, avoiding the join overhead that an equivalent relational schema would require. Second, the **Incident** – **Analysis** relation is a simple reference (**ObjectId**) rather than embedding, because an **Analysis** document is independently queried, listed and resolved outside the incident workflow. Third, **ProjectHealth** aggregates data computed from **Pipeline**, **Analysis** and **Vulnerability** on a schedule rather than being updated transactionally, which is acceptable since health scores are inherently eventually-consistent indicators rather than transactional invariants.

4.5.1 Service-layer class diagram

Beyond the persistence layer, the business logic is implemented by a set of stateless service classes (Figure 4.6). Each service exposes a small set of public operations and depends on one or more Mongoose models; dependencies are drawn as dashed arrows (UML «use» dependency, as opposed to the solid composition/association lines of Figure 4.5).

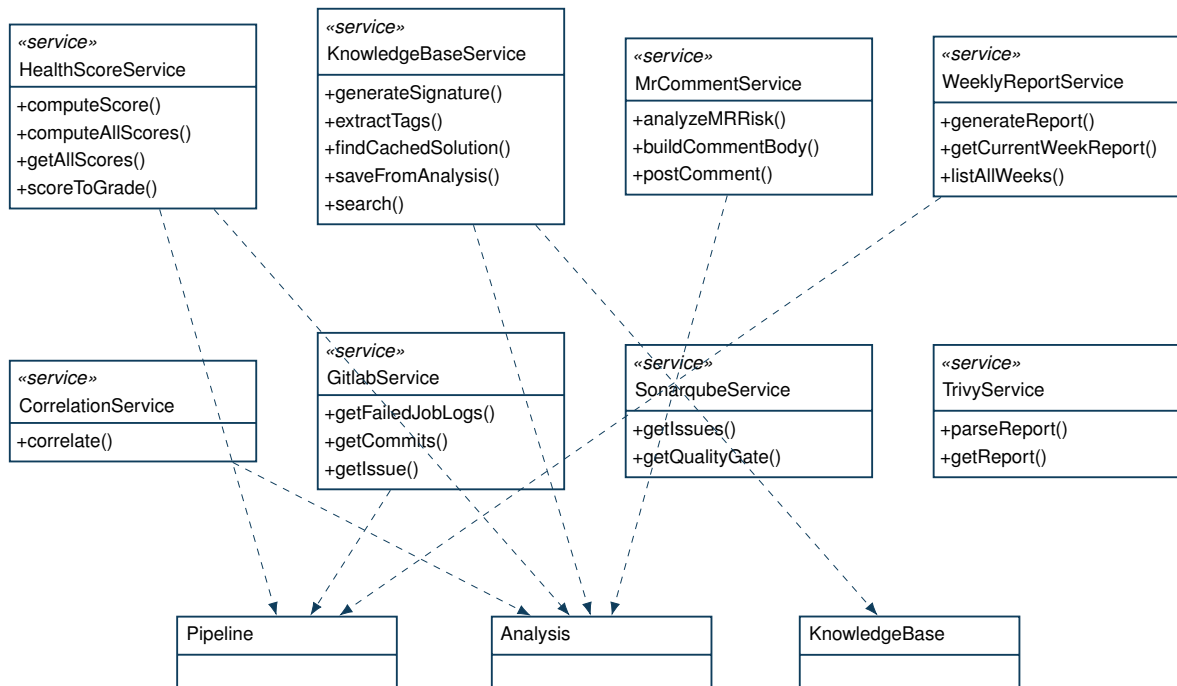


Figure 4.6: Service-layer class diagram and its dependencies on the persistence layer

Listing 4.2 shows the Analysis schema, central to the platform.

```

1  const analysisSchema = new Schema({
2    pipelineId:    { type: String, required: true, index: true },
3    projectId:    { type: String, required: true },
4    errorType:    { type: String, enum: ['build_failure', '
      test_failure',
5                    'dependency_issue', 'security_vulnerability',
6                    'configuration_error', 'unknown'] },
7    rootCause:    String,
8    confidence:   { type: Number, min: 0, max: 1 },
9    riskLevel:    { type: String, enum: ['critical', 'high', 'medium',
      'low'] },
10   suggestedFixes: [suggestedFixSchema], // priority, description,
      command, codeHint
11   affectedFiles: [String],
12   processingTime: Number,
13   resolved:     { type: Boolean, default: false },
14   mtrr:         Number

```

```
15 }, { timestamps: true });
```

Listing 4.2: Analysis Mongoose schema (excerpt)

4.6 Business Services

4.6.1 Health-score service

Each project receives a 0–100 score computed from six weighted metrics, then mapped to a letter grade. Tables 4.3 and 4.4 detail the weighting and the grade mapping.

Table 4.3: Health-score metrics and weights

Metric	Weight	Logic
Pipeline success rate	25%	% of “success” pipelines over 7 days
Critical vulnerabilities	20%	$100 - (\text{critical} \times 20 + \text{high} \times 10)$
Average MTTR	20%	$<15 \text{ min} \rightarrow 100, <30 \rightarrow 80, <60 \rightarrow 60, \text{ else } 30$
Last failure age	10%	$\geq 7\text{d} \rightarrow 100, \geq 3\text{d} \rightarrow 80, \geq 1\text{d} \rightarrow 50, \text{ else } 20$
Test coverage	15%	based on SonarQube data
Code smells	10%	$0 \rightarrow 100, <5 \rightarrow 80, <20 \rightarrow 60, \text{ else } 30$

Table 4.4: Score-to-grade mapping

≥ 90	≥ 75	≥ 60	≥ 45	< 45
A	B	C	D	F

4.6.2 Knowledge-base service

This service stores solutions to recurring errors. Each entry is keyed by a deterministic signature `MD5(errorType + ":" + rootCause[:100])`, normalized to lowercase. When an analyst marks an analysis as resolved, the solution is memorized; on a later similar analysis, a cache hit injects the known solution at the top of the suggested fixes, avoiding a redundant LLM call.

4.6.3 Other services

The *MR-comment service* computes a 0–100 risk score and posts a French explanation on the GitLab merge request. The *weekly-report service* aggregates pipeline counts, success

rate, new vulnerabilities, average MTTR and incident counts, with week-over-week comparison. The *correlation service* links vulnerable packages mentioned in logs to SonarQube issues and failed CI files, producing the context passed to the LLM. Three BullMQ queues handle asynchronous work: `pipeline-analysis` (5 workers, 3 retries), `log-collection` (3 workers) and `notifications` (10 workers).

4.7 REST API Design

The API exposes resource-oriented endpoints protected by JWT and RBAC. Table 4.5 summarizes the main routes.

Table 4.5: Main REST API endpoints

Method	Endpoint	Role	Description
POST	<code>/api/auth/login</code>	—	Login (access 15 min + refresh 7 d)
POST	<code>/api/webhooks/gitlab</code>	HMAC	GitLab pipeline & MR events
GET	<code>/api/pipelines</code>	viewer	Paginated list + filters
GET	<code>/api/pipelines/stats</code>	viewer	Dashboard KPIs (7 days)
POST	<code>/api/pipelines/:id/retry</code>	analyst	Re-trigger analysis
GET	<code>/api/analyses/recurring</code>	viewer	Top 5 recurring issues
PATCH	<code>/api/analyses/:id/resolve</code>	analyst	Mark resolved (feeds KB)
GET	<code>/api/vulnerabilities</code>	viewer	CVE list (filters)
GET	<code>/api/knowledge/search</code>	viewer	Full-text KB search
GET	<code>/api/health-score</code>	viewer	All project scores
GET	<code>/api/incidents/:id</code>	viewer	Incident detail + timeline
POST	<code>/api/incidents/:id/postmortem</code>	analyst	Generate AI post-mortem
GET	<code>/api/reports/current</code>	viewer	Current-week report

4.7.1 API design rationale

Three design decisions shape every endpoint of the API. First, **pagination is mandatory** on every list endpoint (`?page=`, `?limit=`, capped at 100) to avoid unbounded MongoDB scans as the number of pipelines grows over time. Second, **errors follow a single envelope** `{ error: { code, message } }` with a stable machine-readable code (e.g. `INVALID_SIGNATURE`, `NOT_FOUND`, `FORBIDDEN`), so that the frontend can branch on `code` rather than parsing free-text messages. Third, the webhook ingestion endpoint deliberately **returns 202 Accepted** rather than 200/201: the HTTP response

only confirms that the event was queued, not that the analysis is complete, which keeps GitLab’s webhook delivery (which times out after 10 s) decoupled from the LLM call latency.

4.7.2 Database design rationale

MongoDB was chosen over a relational database for three reasons specific to this domain. First, the **Analysis** document’s shape *varies* with the error type (a **security_vulnerability** analysis carries CVE references that a **build_failure** analysis never has); a flexible schema avoids a wide table with mostly-null columns or a fragile EAV (Entity-Attribute-Value) relational pattern. Second, the embedded sub-documents identified in Figure 4.5 (**suggestedFixes**, **timeline**, **breakdown**) are always read together with their parent and never queried independently, which is the textbook case for document embedding versus normalization. Third, the write pattern is append-heavy and time-ordered (new pipelines, new analyses), which plays to MongoDB’s strengths with capped growth and TTL-friendly collections, should historical pruning be needed later.

Indexing follows the query patterns actually issued by the dashboard: **Pipeline** is indexed on (**projectId**, **createdAt**) for the per-project timeline and on **pipelineId** for webhook lookups; **Analysis** is indexed on **pipelineId** (unique lookup from the pipeline detail page) and on **errorType** (recurring-issues aggregation); **KnowledgeBase** is indexed on the unique **errorSignature** field, since every cache lookup is an exact-match query on that field; **ProjectHealth** is indexed on the unique **projectId**, since only the latest score per project is read on the dashboard.

4.8 Additional Behavioral Views

Beyond the pipeline-analysis sequence diagram of Figure 4.4, two other use cases identified in Chapter 3 involve a non-trivial sequence of calls across services and are detailed below.

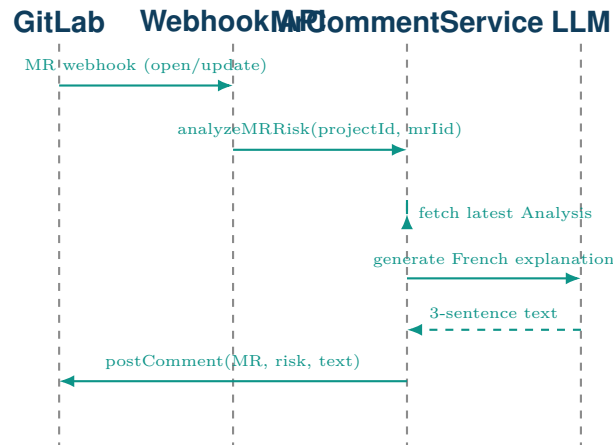


Figure 4.7: Sequence diagram: automatic merge-request risk comment

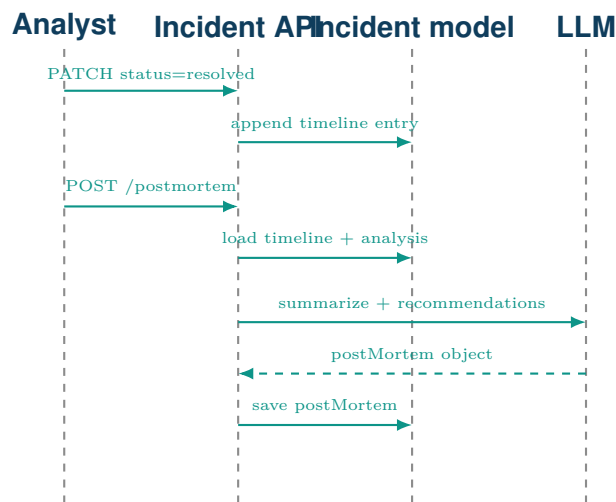


Figure 4.8: Sequence diagram: incident resolution and AI post-mortem generation

4.9 Frontend Architecture

The frontend is a React (Vite) single-page application. Global state is split between **Redux Toolkit** (authentication tokens, active filters), **TanStack Query** (server-state cache, queries and mutations with cache invalidation) and a **Socket.io** client for real-time events. Received events such as `analysis:complete`, `pipeline:new` and `incident:created` invalidate the relevant queries, refreshing the dashboard instantly. The application is structured into eleven pages (Dashboard, Pipelines, Analyses, Security, Health Score, Incidents, Knowledge Base, Weekly Report, Settings, etc.) and reusable component families (layout, dashboard, analysis, health, incident, knowledge, report, security, generic UI).

4.10 Conclusion

This chapter detailed the design of the solution. The next chapter presents its implementation, testing and deployment.

Chapter 5

Implementation, Testing and Deployment

5.1 Introduction

This chapter describes the working environment, the technology stack, the main interfaces, representative code excerpts, the automated tests, the containerization and Kubernetes deployment, the Jenkins CI/CD pipeline, security measures and the results obtained.

5.2 Working Environment

5.2.1 Hardware environment

Development was carried out on a workstation running **Windows 11** (build 26200), equipped with an **Intel Core i5-11400H** processor (6 cores / 12 threads, 2.70 GHz) and 16 GB of RAM. All infrastructure services requiring a Linux host — GitLab CE, GitLab Runner, Jenkins, the k3s Kubernetes cluster and SonarQube — run inside a dedicated **Vagrant/VirtualBox virtual machine** (Ubuntu 22.04 LTS, 8 GB RAM allocated, 4 virtual CPUs), isolating the lab environment from the host machine.

5.2.2 Software environment

The main tools are Visual Studio Code as the primary IDE, Git 2.47 and GitHub for source control, Node.js 20 LTS (runtime for both backend and frontend tooling) and npm as package manager, Docker (Engine 29.x) and the Docker Compose plugin for containerization, Vagrant and VirtualBox to provision the lab VM, GitLab CE and GitLab Runner for source hosting and pipeline triggers, Jenkins LTS for CI/CD orchestration, SonarQube Community Edition for static analysis, Trivy for container vulnerability scanning, Helm 3 and kubectl for Kubernetes operations, and ngrok to expose the local backend to GitLab webhooks during development.

5.3 Technology Stack

Tables 5.1, 5.2 and 5.3 recap the stack.

Table 5.1: Backend stack

Technology	Role
Node.js 20 / Express	ESM runtime and REST API framework
MongoDB / Mongoose	Document database and ODM
Redis / BullMQ	Broker, rate limiting and asynchronous jobs
Socket.io	Real-time WebSocket communication
Groq / Anthropic	Primary LLM (Llama 3.3 70B) and backup (Claude)
JWT / Bcrypt	Authentication and password hashing
Helmet / Winston	HTTP security and structured logging
Vitest	Unit-testing framework

Table 5.2: Frontend stack

Technology	Role
React 18 / Vite	UI framework and build tool
React Router	SPA routing
TanStack Query	Server-state cache and synchronization
Redux Toolkit	Global state (auth, filters)
Tailwind CSS	Utility-first styling
Recharts	Charts (line, bar, radar)
Socket.io Client	Real-time updates

Table 5.3: Infrastructure and DevOps stack

Component	Role
Docker / Docker-Hub	Containerization and public image registry
k3s (Kubernetes)	Lightweight container orchestrator (Vagrant VM)
Helm 3	Kubernetes package manager
Jenkins LTS	CI/CD server with a declarative pipeline
GitLab CE / Runner	Local Git server and CI/CD
SonarQube CE / Trivy	Static analysis and CVE scanning
ngrok	HTTPS tunnel for local GitLab webhooks

5.4 Application Interfaces

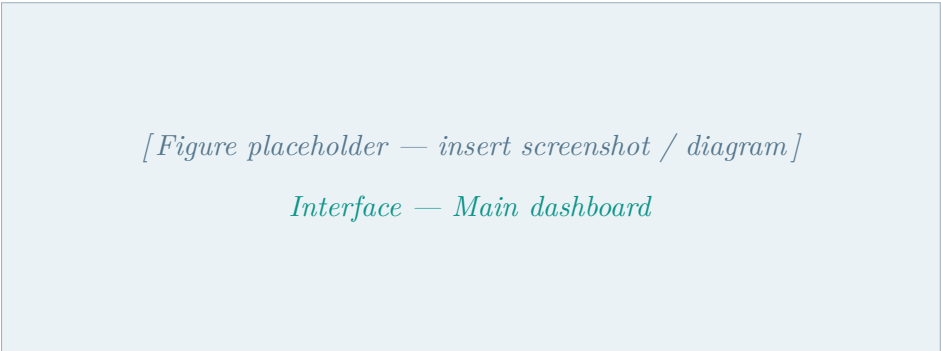


Figure 5.1: Interface — Main dashboard

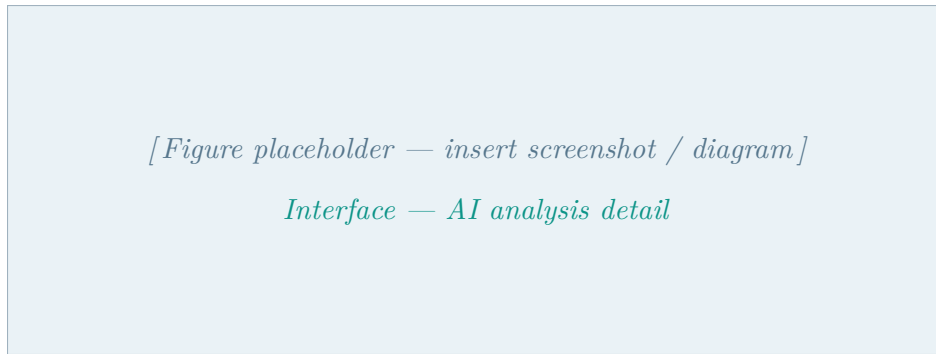


Figure 5.2: Interface — AI analysis detail

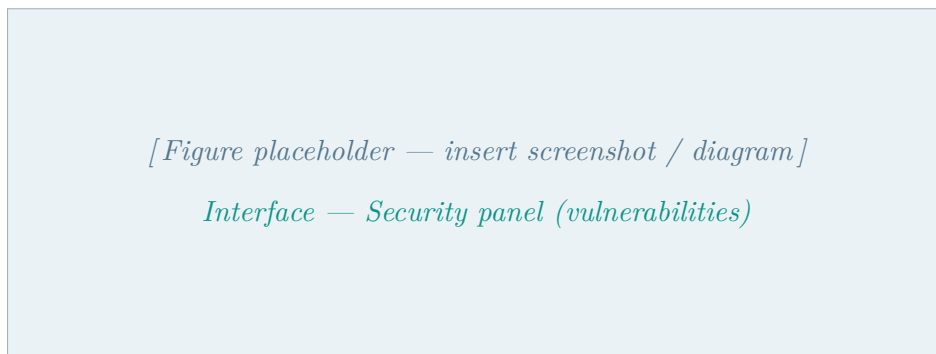


Figure 5.3: Interface — Security panel (vulnerabilities)

To complete / adapt

Replace these three placeholders with real screenshots of the running application: (1) the `/dashboard` page showing KPIs and the 7-day trend chart, (2) the `/pipelines/:id` page showing an AI analysis card with root cause and suggested fixes, (3) the `/security` page showing the Trivy vulnerability table. Use `\includegraphics[width=0.9\textwidth]{img/dashboard.png}` inside the figure environment.

5.5 Key Code Excerpts

5.5.1 Webhook signature validation

Webhook reception is secured by an HMAC-SHA256 signature, compared in constant time to prevent timing attacks.

```
1 const crypto = require("crypto");
2
3 function verifyGitlabSignature(req, res, next) {
4   const received = Buffer.from(req.headers["x-gitlab-token"] || "")
5   ;
```

```

5   const expected = Buffer.from(crypto
6     .createHmac("sha256", process.env.GITLAB_WEBHOOK_SECRET)
7     .update(req.rawBody)
8     .digest("hex"));
9   if (received.length !== expected.length ||
10      !crypto.timingSafeEqual(received, expected)) {
11     return res.status(401).json({ error: "Invalid signature" });
12   }
13   next();
14 }

```

Listing 5.1: HMAC validation middleware (excerpt)

5.5.2 Weighted health-score computation

```

1  function computeHealthScore(m) {
2    const score =
3      0.25 * m.pipelineSuccessRate
4      + 0.20 * m.criticalVulnsScore
5      + 0.20 * m.mttrScore
6      + 0.15 * m.coverageScore
7      + 0.10 * m.lastFailureScore
8      + 0.10 * m.codeSmellsScore;
9    return scoreToGrade(Math.round(score)); // 0..100 -> A..F
10 }

```

Listing 5.2: Weighted health-score computation (excerpt)

5.5.3 BullMQ worker with bounded concurrency and retries

The analysis worker is registered once at startup with a fixed concurrency and a retry policy with exponential backoff, so that a transient failure of an external API (GitLab, SonarQube) does not fail the whole job permanently.

```

1  const worker = new Worker('pipeline-analysis', async (job) => {
2    const { pipelineId, projectId } = job.data;
3    const logs = await gitlabService.getFailedJobLogs(projectId,
4      pipelineId);
5    const sonar = await sonarqubeService.getIssues(projectId);
6    const trivy = await trivyService.getReport(projectId, pipelineId)
7      ;
8    const normalized = normalizerService.normalize({ logs, sonar,
9      trivy });
10
11    const cached = await knowledgeBaseService.findCachedSolution(
12      normalized);

```

```

9   const result = cached ?? await rootCauseAnalyzer.analyze(
    normalized);
10
11   await Analysis.create({ pipelineId, projectId, ...result });
12   await healthScoreService.computeScore(projectId);
13   io.emit('analysis:complete', { pipelineId, projectId });
14 }, {
15   connection: redisConnection,
16   concurrency: 5,
17 });
18
19 worker.on('failed', (job, err) => logger.error('Job ${job.id}
    failed', err));

```

Listing 5.3: Pipeline-analysis worker registration (excerpt)

5.5.4 Resilient MongoDB connection

The backend retries its initial MongoDB connection with a capped exponential backoff instead of crashing, because in a Kubernetes deployment the `mongodb` pod may not yet be ready when the `aiops-backend` pod starts (no explicit `initContainer` ordering is configured in the current Helm chart).

```

1  async function connectWithRetry(uri, attempt = 1) {
2    try {
3      await mongoose.connect(uri, { serverSelectionTimeoutMS: 5000 })
4      ;
5      logger.info('MongoDB connected');
6    } catch (err) {
7      const delay = Math.min(1000 * 2 ** attempt, 30000);
8      logger.warn('MongoDB connection failed (attempt ${attempt}),
9        retrying in ${delay}ms');
10     setTimeout(() => connectWithRetry(uri, attempt + 1), delay);
11   }
12 }

```

Listing 5.4: Mongoose connection with retry (excerpt)

5.5.5 Real-time data fetching on the frontend

On the frontend, TanStack Query combines polling with cache invalidation triggered by Socket.io events, so the dashboard stays fresh without naive fixed-interval polling once a WebSocket connection is established.

```

1  export function usePipelines(filters) {
2    const queryClient = useQueryClient();

```

```

3
4   useEffect(() => {
5       const socket = getSocket();
6       const onNewPipeline = () => queryClient.invalidateQueries(['
           pipelines']);
7       socket.on('pipeline:new', onNewPipeline);
8       return () => socket.off('pipeline:new', onNewPipeline);
9   }, [queryClient]);
10
11   return useQuery({
12       queryKey: ['pipelines', filters],
13       queryFn: () => pipelinesApi.getAll(filters),
14       staleTime: 30_000,
15   });
16 }

```

Listing 5.5: usePipelines hook with Socket.io-driven cache invalidation (excerpt)

5.6 Automated Testing

Unit tests rely on **Vitest** (native ESM, integrated with Vite). The suite comprises 25 tests, summarized in Table 5.4.

Table 5.4: Automated test suite

Test file	Tests	Scope
healthScore.service.test.js	12	scoring functions, grade mapping, edge cases
knowledgeBase.service.test.js	6	signature determinism, tag extraction
Incidents.test.jsx	3	timeSince formatting
HealthScore.test.jsx	2	buildHistory aggregation
Spinner.test.jsx	2	React component rendering
Total	25	100% passing

The backend suite (18 tests) runs in under seven seconds; the frontend suite (7 tests)

uses `@testing-library/react` and `jsdom`. Both suites are executed automatically inside the Jenkins pipeline.

5.6.1 Test strategy and pyramid

The 25 automated tests target the lower, fastest layer of the classic test pyramid: pure functions and isolated components with no network or database dependency (Figure 5.4). This was a deliberate scoping decision under the time constraints of a solo final-year project: unit tests on scoring logic and signature generation catch the highest-risk regressions (an incorrect health-score weight or a non-deterministic knowledge-base signature would silently corrupt data for every project) at near-zero cost, whereas integration tests against a real MongoDB/Redis and end-to-end browser tests would have required a dedicated test environment and more CI time than was available. This trade-off, and the concrete plan to close the gap, is revisited explicitly in the Critical Discussion chapter.

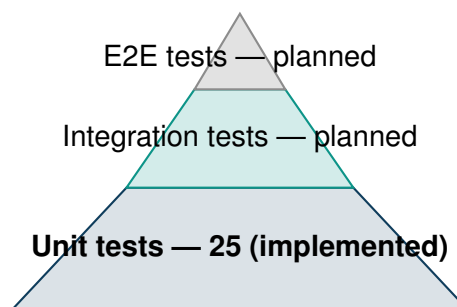


Figure 5.4: Test pyramid: implemented coverage vs. planned extensions

5.6.2 What is intentionally not covered

Table 5.5 lists the categories of behavior left out of the automated suite, together with the manual verification actually performed as a substitute and the risk this leaves open.

Table 5.5: Coverage gaps and manual mitigation

Not automated	Manual mitigation performed / residual risk
HMAC webhook validation under real GitLab payloads	Manually tested with live GitLab webhooks during 5 sprints; risk: payload shape drift in a GitLab upgrade would not be caught automatically
BullMQ worker concurrency & retry behavior	Manually triggered failures via forced API timeouts; risk: race conditions under very high concurrency are unverified
React component integration (page-level)	Manually exercised every page in the browser each sprint; risk: visual regressions are not caught automatically
Helm chart rendering & Kubernetes rollout	Verified via <code>helm template</code> + manual <code>kubectl rollout status</code> on every deploy; risk: a <code>values.yaml</code> typo could pass review unnoticed

5.7 Containerization

Each service ships as a Docker image. The frontend uses a multi-stage build that compiles the SPA and serves it with nginx, which also reverse-proxies API and WebSocket traffic to the backend service.

```

1 FROM node:20-alpine AS builder
2 WORKDIR /app
3 COPY package*.json ./
4 RUN npm ci
5 COPY . .
6 RUN npm run build          # -> dist/ (static SPA)
7
8 FROM nginx:alpine
9 COPY --from=builder /app/dist /usr/share/nginx/html
10 COPY nginx.conf /etc/nginx/conf.d/default.conf
11 EXPOSE 80
12 CMD ["nginx", "-g", "daemon off;"]

```

Listing 5.6: Frontend multi-stage Dockerfile

5.8 Kubernetes Deployment with Helm

The platform is deployed on a lightweight Kubernetes cluster (k3s) through a Helm chart. The chart defines deployments for the backend, frontend (nginx) and MongoDB, a Redis StatefulSet, ClusterIP services, persistent storage for MongoDB, and a Traefik ingress. Sensitive values (JWT secret, GitLab token, API keys) are kept in a git-ignored `values.secrets.yaml` and injected as Kubernetes secrets. Figure 5.5 shows the resulting cluster topology.

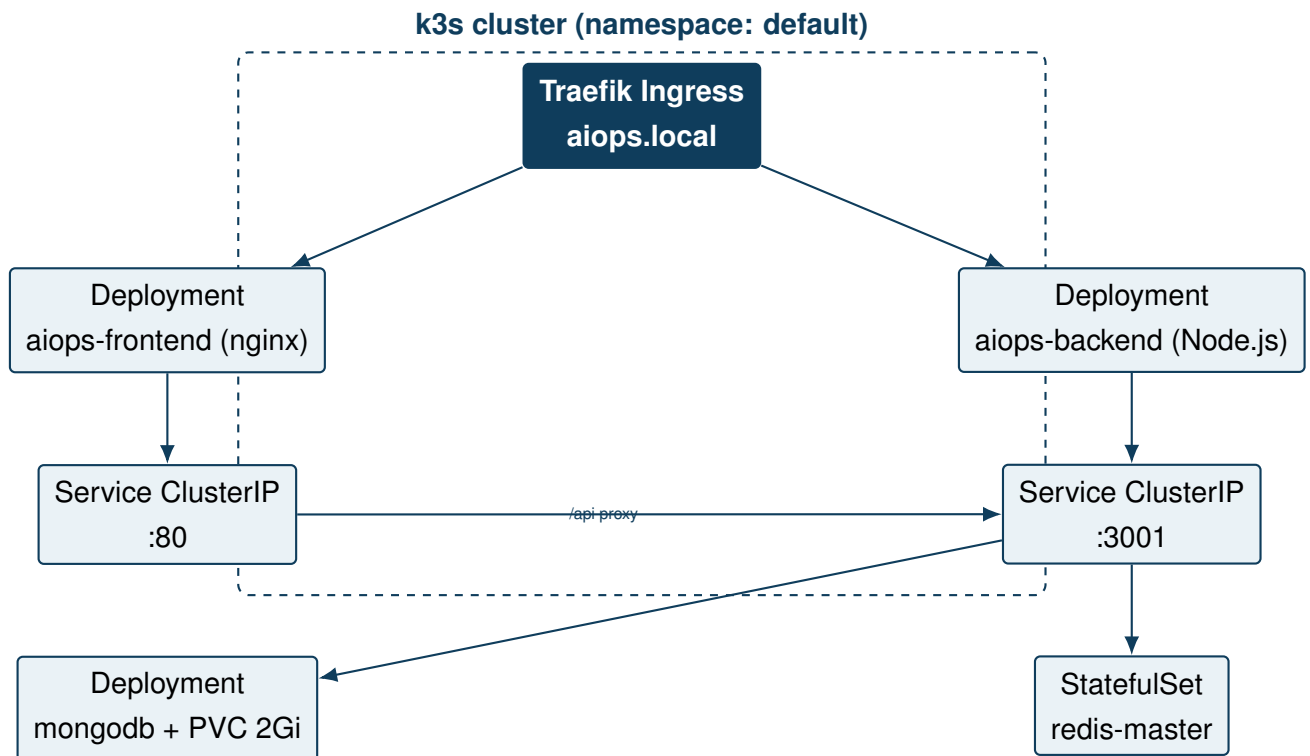


Figure 5.5: Kubernetes (k3s) deployment architecture

```

1 backend:
2   image: { repository: khalil512/aiops-backend, tag: latest }
3   replicas: 1
4   port: 3001
5   resources:
6     requests: { cpu: 100m, memory: 256Mi }
7     limits:   { cpu: 500m, memory: 512Mi }
8   config:
9     nodeEnv:    production
10    mongodbUri: mongodb://mongodb:27017/aiops

```

```

11   redisHost:  redis-master
12   groqModel:  llama-3.3-70b-versatile
13 ingress:
14   enabled: true
15   className: traefik
16   host: aiops.local

```

Listing 5.7: Helm values (excerpt)

5.9 Jenkins CI/CD Pipeline

A declarative Jenkins pipeline automates the full delivery chain. Jenkins runs as a container with access to the Docker daemon, `kubectl` and `helm`, and to the k3s kubeconfig. Listing 5.8 shows the pipeline stages (condensed) and Figure 5.6 the corresponding end-to-end flow.

```

1 pipeline {
2   agent any
3   environment { DOCKERHUB_TOKEN = credentials('dockerhub-token') }
4   stages {
5     stage('Install & test backend') { steps { sh 'cd backend &&
        npm ci && npm test' } }
6     stage('Install & test frontend') { steps { sh 'cd frontend &&
        npm ci && npm test' } }
7     stage('Build images') {
8       steps {
9         sh 'docker build -t khalil512/aiops-backend:latest backend'
10        sh 'docker build -t khalil512/aiops-frontend:latest
            frontend'
11      }
12    }
13    stage('Push images') {
14      steps { sh 'echo $DOCKERHUB_TOKEN | docker login -u khalil512
        --password-stdin && docker push khalil512/aiops-backend:
        latest && docker push khalil512/aiops-frontend:latest' }
15    }
16    stage('Deploy to k3s') {
17      steps { sh 'helm upgrade aiops ./helm/aiops -f values.yaml -f
        $SECRETS_FILE --wait --timeout 5m && kubectl rollout
        restart deployment/aiops-backend deployment/aiops-frontend
        ' }
18    }
19  }
20 }

```

Listing 5.8: Jenkinsfile stages (condensed)

The pipeline checks out the source from GitHub, installs dependencies, runs the 25 tests, builds and pushes the two Docker images to DockerHub, then upgrades the Helm release and restarts the deployments on k3s. A full run takes about fifteen minutes initially and ten minutes on subsequent runs thanks to Docker layer caching.

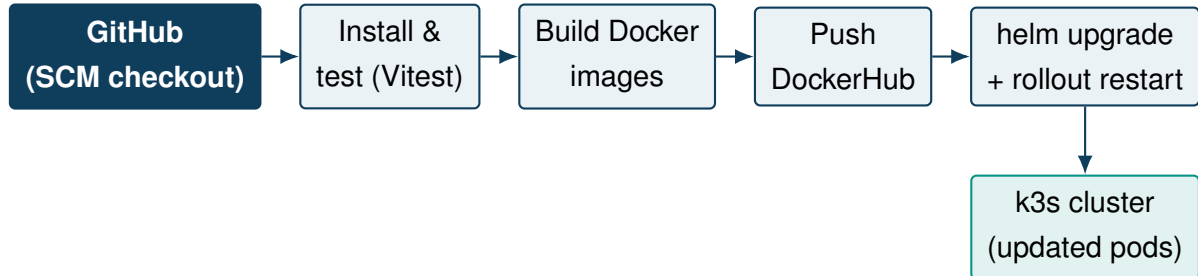


Figure 5.6: End-to-end CI/CD flow

5.10 Engineering Challenges and Solutions

Setting up a self-hosted GitLab/Jenkins/k3s lab environment surfaced several non-obvious infrastructure issues whose resolution is documented here, both as evidence of the troubleshooting effort involved and as a reference for reproducing the environment. Table 5.6 summarizes the most instructive ones.

Table 5.6: Selected infrastructure issues encountered and their resolution

Symptom	Root cause and fix
Jenkins container could not reach the k3s API server (connection refused on 127.0.0.1:6443)	The kubeconfig pointed to 127.0.0.1, which inside a bridged Docker network resolves to the <i>container's own</i> loopback, not the VM host's. Fixed by recreating the Jenkins container with <code>--network host</code> so it shares the VM's network namespace.
npm: not found when the Jenkins pipeline ran npm test	The official <code>jenkins/jenkins:lts</code> image ships no Node.js runtime. Fixed by installing Node.js 20 directly inside the running container via the NodeSource setup script; documented as non-persistent across container recreation, requiring re-installation if the container is ever recreated.
Helm upgrade succeeded but pods kept serving the old code	With a <code>:latest</code> image tag, the Deployment spec is byte-identical after <code>helm upgrade</code> , so Kubernetes never detects a change and skips the rollout. Fixed by chaining <code>kubectl rollout restart</code> immediately after every <code>helm upgrade</code> in the Jenkinsfile.
Jenkins REST API calls returned 403 No valid crumb	Git Bash and Node.js resolve <code>/tmp</code> to different physical paths on Windows, so the CSRF crumb fetched by one tool was invisible to the other. Fixed by storing the crumb and cookie jar in a path on the shared Windows filesystem, accessed identically by both tools.
Kubernetes Dashboard unreachable through the VirtualBox port forward despite kubectl proxy running	The proxy was bound to 127.0.0.1 inside the VM; VirtualBox's NAT port forwarding delivers traffic through the guest's virtual network interface, which never reaches a service bound to loopback only. Fixed by rebinding the proxy to 0.0.0.0 as a persistent <code>systemd</code> service, surviving VM reboots.

These five issues share a common pattern: each stems from an implicit assumption

about network or filesystem scope (container vs. host loopback, Windows tool-specific path resolution, Kubernetes change detection) that only became visible once the full stack was exercised end-to-end — reinforcing the value of the working, deployed system as the actual validation target, rather than each component tested in isolation.

5.11 Security

Security is addressed at several layers, summarized in Table 5.7.

Table 5.7: Security measures

Layer	Measure
Authentication	JWT access (15 min) + refresh (7 d), three-level RBAC
Webhooks	HMAC-SHA256 with constant-time comparison
Passwords	Bcrypt hashing (12 salt rounds)
HTTP	Helmet (CSP, HSTS, X-Frame-Options)
Rate limiting	Redis store per endpoint (webhook 30/min, API 100/min)
Secrets	<code>.env</code> and <code>values.secrets.yaml</code> excluded from version control
Kubernetes	Opaque secrets injected via Helm

5.12 Results

5.12.1 Performance metrics

Table 5.8: Performance results

Indicator	Result
AI analysis time (webhook to result)	< 10 seconds
Error-type classification accuracy	≈ 85% (tested cases)
Diagnosis-time reduction	30–90 min → < 2 min (×20)
BullMQ processing time (without LLM)	< 1 second
WebSocket latency (webhook to dashboard)	< 500 ms

5.12.2 End-to-end validation

A complete end-to-end test was validated under real conditions: a `git push` triggers a GitLab pipeline; the webhook reaches the backend in under a second; a BullMQ job collects the CI logs; Groq performs the analysis; the result is stored in MongoDB; the knowledge base and health score are updated; and the React dashboard refreshes in real time over WebSocket — the whole chain completing in seven to ten seconds.

5.12.3 Validated features

All targeted features were implemented and validated: secured webhook reception, AI root-cause analysis, GitLab/SonarQube/Trivy integration, self-feeding knowledge base, per-project health score, full incident lifecycle with AI post-mortems, automatic MR risk comments, weekly reports, real-time dashboard, JWT/RBAC security, the 25-test suite, multi-stage Docker builds, Helm deployment on k3s, and the complete Jenkins CI/CD pipeline.

5.13 Conclusion

This chapter presented the concrete implementation, testing and deployment of the platform, validating the objectives set in the first chapter. Honest engineering, however, requires examining where the solution falls short of an ideal, production-grade system — which is the purpose of the next chapter.

Chapter 6

Critical Discussion, Limitations and Mitigations

6.1 Introduction

A defensible final-year project should not only present what works, but also reason explicitly about what does not yet, and why. This chapter anticipates the questions a review jury is most likely to raise about the platform described in Chapters 3 and 4, organized by category, together with the mitigation already in place or planned.

6.2 Methodological Limitations

6.2.1 AI classification accuracy measured on a limited sample

The 85% error-type classification accuracy reported in Table 5.8 was measured on a few dozen real pipeline failures observed during the project’s sprints, not on a statistically representative, labeled benchmark. This is a methodological weakness: the figure should be read as an encouraging early signal, not a validated production SLA. A rigorous follow-up would build a labeled dataset of at least a few hundred failures across multiple projects and report precision/recall per `errorType` rather than a single aggregate accuracy.

6.2.2 No formal usability study

The dashboard’s usability was validated informally (the author’s own use, and brief feedback from the host team) rather than through a structured usability test with target engineers. A System Usability Scale (SUS) questionnaire administered to the DevOps unit would strengthen the claim that the tool measurably reduces cognitive load, beyond the timing measurements of Table 1.3.

6.3 Architectural Limitations

6.3.1 Single replica, no high availability

Every Deployment in the current Helm chart (Table 5.1, Section 5.3) runs a single replica. The backend is, however, designed *stateless* by construction — session state lives in JWTs, not in process memory, and job state lives in Redis/BullMQ — so horizontal scaling to replicas: N is a one-line `values.yaml` change with no code modification, as already verified manually (Table 3.1, scalability row). What is genuinely missing is a `HorizontalPodAutoscaler` and a load balancer health-check tuned for the `/api/health` endpoint, both straightforward additions deferred for time reasons.

6.3.2 Single MongoDB instance, no replica set or backup policy

MongoDB runs as a single pod with a single PVC and no replica set, meaning a node failure or PVC corruption would cause data loss. The mitigation path is well understood (a 3-node replica set via the Bitnami MongoDB Helm chart, plus a scheduled `mongodump` CronJob to object storage) but was not implemented within the project's time budget, since the lab cluster has a single VM/node and a replica set's benefit is mostly demonstrative in that topology.

6.3.3 Secrets management without a vault

Sensitive values (JWT secret, GitLab token, LLM API keys) are kept in a git-ignored `values.secrets.yaml` injected as Kubernetes `Opaque` secrets (base64-encoded, not encrypted at rest by default in etcd). A production deployment should instead use a secret manager (HashiCorp Vault, or Sealed Secrets / SOPS for a GitOps-friendly alternative) so that secrets are encrypted both at rest and in the Git history of the deployment repository, if one were used.

6.4 Data Privacy and Third-Party LLM Dependency

Every LLM call sends a context excerpt (log lines, commit messages, SonarQube issue descriptions) to a third-party API (Groq, with Claude as backup). For the lab projects used during this PFE this is an acceptable trade-off, but it would not be acceptable as-is for a client project under strict data-residency or confidentiality clauses. Three mitigations are already partially in place or planned: (1) the correlation/normalization step (Section 4.4) only forwards the *relevant excerpt* of a failing job's log, not the entire raw log, which already limits exposure; (2) the knowledge-base cache (Section 4.6.2) means a recurring error is diagnosed by the LLM only once, after which the memorized

solution is reused without any further data leaving the platform; (3) the dual-provider design (Groq primary, Claude backup) could be extended with a third, self-hosted option (e.g. a quantized Llama model served via Ollama inside the cluster) for clients who require on-premises inference, at the cost of classification quality — a trade-off to be validated, not assumed.

6.5 Operational and Process Limitations

- **Manual CI/CD trigger:** the Jenkins pipeline is currently started via “Build Now” rather than an automatic GitHub webhook, because the lab Jenkins instance is not exposed on a public endpoint reachable from GitHub. The Jenkinsfile and pipeline logic are already webhook-ready; only the trigger configuration is missing.
- **Coarse-grained RBAC:** the three roles (admin/analyst/viewer) are global, not scoped per project. A client onboarding several teams with different project visibility would need project-level permissions, not implemented in the current model.
- **Limited automated test coverage:** as discussed in Section 5.6.2, only the unit layer of the test pyramid is automated; integration and end-to-end tests are identified but not yet implemented.
- **No production monitoring of the platform itself:** the AIOps platform monitors *client* CI/CD pipelines, but its own health (API latency, worker queue depth, LLM error rate) is not yet exported to Prometheus/Grafana, which is an ironic but honest gap for an observability-oriented tool.

6.6 Anticipated Jury Questions

Table 6.1 compiles, in a direct question/answer format, the critiques most likely to be raised during the defense, consolidating the discussion above into quick-reference answers.

Table 6.1: Anticipated jury questions and prepared answers

Question	Prepared answer
Why trust an LLM’s root-cause analysis instead of a deterministic rule engine?	The error types are too varied for an exhaustive rule set; the LLM is constrained with a structured prompt and its output is always a suggestion reviewed by an analyst, never an automatic action, and the knowledge base captures validated solutions to reduce reliance on the LLM over time.
What happens if Groq is down?	The Anthropic Claude client is wired as an automatic fallback in <code>ai/</code> ; if both are unavailable, the job retries (BullMQ backoff) and ultimately surfaces as a failed analysis rather than blocking the pipeline itself.
Is this production-ready?	The application layer is; the infrastructure layer (single replica, single Mongo instance, vault-less secrets) is lab-grade and the gap to production is explicitly mapped in this chapter, not hidden.
How do you know the 85% accuracy figure is meaningful?	It is not a validated benchmark, only an early signal from manual review of project runs; Section 6.2.1 states this limitation explicitly and proposes a labeled-dataset follow-up.
Why MongoDB and not PostgreSQL?	The document shape varies by error type and several fields are always read/written together with their parent; the rationale is detailed in Section 4.7.2.

6.7 Conclusion

None of the limitations discussed in this chapter invalidate the core contribution — a working, end-to-end, AI-assisted CI/CD analysis platform deployed on Kubernetes with

a full CI/CD pipeline of its own — but acknowledging them precisely is what turns the project from a demo into an engineering artifact whose next iteration is already mapped out.

Chapter 7

Project Management

7.1 Introduction

This chapter documents how the project was planned and tracked as a solo, time-boxed final-year project, complementing the Scrum-inspired sprint breakdown already introduced in Section 1.6 (Figure 1.2) with a work breakdown structure and a risk register.

7.2 Work Breakdown Structure

Figure 7.1 decomposes the project into four work packages aligned with the four implementation chapters of this report, each further split into the concrete deliverables produced.

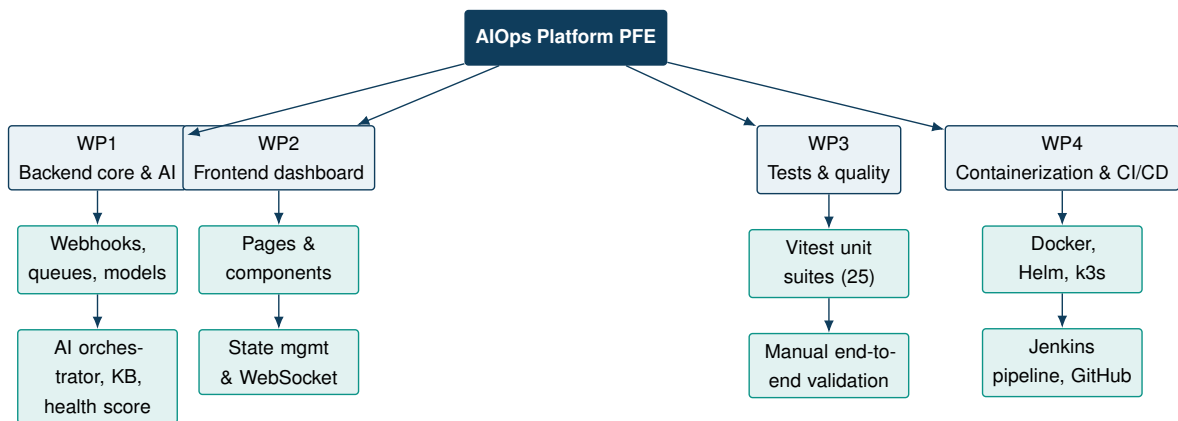


Figure 7.1: Work breakdown structure (WBS)

7.3 Risk Register

Table 7.1 lists the technical risks identified at the start of the project, rated on a 1–3 likelihood/impact scale, with the mitigation actually applied. Two risks materialized during the project (marked *occurred*): the VM resource contention risk, observed when GitLab CE, Jenkins and k3s ran concurrently on the same 4-vCPU VM and caused a load average of 25, matching exactly the troubleshooting log of Section 5.10.

Table 7.1: Risk register

Risk	Likelihood	Impact	Mitigation
LLM API outage or rate limiting	2	2	Dual-provider fallback (Groq + Claude), retries with backoff
VM resource contention (Git-Lab+Jenkins+k3s)	3	2	Lightweight images, sequential pipeline stages; <i>occurred</i> (Section 5.10)
Scope creep (9 ambitious objectives, solo developer)	3	3	Strict sprint boundaries (Figure 1.2), MVP-first ordering
Windows/Linux tooling friction (path resolution)	2	1	Shared-path workaround for crumb auth; <i>occurred</i> (Section 5.10)
Knowledge loss between sprints (solo developer, long gaps)	1	2	This written report itself, plus inline code documentation

7.4 Effort Distribution

Table 7.2 gives an approximate distribution of effort across the project’s macro-activities, self-reported at the end of each sprint.

Table 7.2: Approximate effort distribution

Activity	Share of effort
Backend development (API, queues, AI orchestration)	30%
Frontend development (React dashboard)	20%
Infrastructure (Vagrant, GitLab, Jenkins, k3s, Helm)	25%
Testing and debugging	15%
Documentation and this report	10%

7.5 Conclusion

This chapter completes the engineering narrative of the report with the project-management view: how the scope was broken down, which risks were anticipated and which materialized, and where the effort actually went. The general conclusion follows.

General Conclusion and Future Work

The AIOps platform developed in this final year project addresses a concrete problem faced daily by the DevOps teams at Capgemini — Altran Telnet Corporation Tunisia. Beyond simple pipeline diagnosis, it constitutes a complete operational-intelligence system: automatic root-cause analysis in under ten seconds, organizational memory through the self-feeding knowledge base, continuous quality steering via health scores, structured incident management with traceability and learning, and prevention through merge-request risk analysis.

The measured reduction in diagnosis time — from 30–90 minutes to under two minutes — represents a significant productivity gain, reducing the cognitive load of incident handling and freeing engineers for higher-value tasks. Entirely containerized and deployed on Kubernetes with a Jenkins CI/CD pipeline, the solution is production-ready and extensible.

On a personal level, this work consolidated skills in full-stack development, event-driven architecture, LLM integration and DevOps/Cloud practices. Several future directions emerge:

- active alerting (email, Slack, PagerDuty) on critical incidents;
- predictive ML estimating pipeline failure probability before execution;
- support for other CI/CD platforms (GitHub Actions, Azure DevOps);
- Prometheus metrics export for Grafana dashboards;
- multi-tenant isolation with per-team quotas.

Bibliography

- [1] Gartner, *AIOps (Artificial Intelligence for IT Operations) Platforms*, IT Glossary.
- [2] GitLab Documentation, *Webhooks and CI/CD Pipelines*, <https://docs.gitlab.com>.
- [3] SonarSource, *SonarQube Documentation*, <https://docs.sonarsource.com>.
- [4] Aqua Security, *Trivy Vulnerability Scanner*, <https://trivy.dev>.
- [5] BullMQ, *Premium Message Queue for NodeJS*, <https://docs.bullmq.io>.
- [6] Groq, *GroqCloud API Reference*, <https://console.groq.com/docs>.
- [7] Rancher, *k3s — Lightweight Kubernetes*, <https://k3s.io>.
- [8] The Helm Authors, *Helm Documentation*, <https://helm.sh/docs>.
- [9] Jenkins, *Pipeline Documentation*, <https://www.jenkins.io/doc>.
- [10] Meta, *React Documentation*, <https://react.dev>.

Appendix A

Full Helm Values File

Listing A.1 reproduces the complete `values.yaml` used to deploy the platform on k3s (the companion `values.secrets.yaml`, holding the JWT secret, GitLab token and LLM API keys, is git-ignored and intentionally not reproduced here).

```
1 backend:
2   image:
3     repository: khalil512/aiops-backend
4     tag: latest
5   replicas: 1
6   port: 3001
7   resources:
8     requests: { cpu: 100m, memory: 256Mi }
9     limits:   { cpu: 500m, memory: 512Mi }
10
11 frontend:
12   image:
13     repository: khalil512/aiops-frontend
14     tag: latest
15   replicas: 1
16   port: 80
17   resources:
18     requests: { cpu: 50m, memory: 64Mi }
19     limits:   { cpu: 200m, memory: 128Mi }
20
21 mongodb:
22   image: mongo:4.4
23   persistence:
24     size: 2Gi
25   resources:
26     requests: { cpu: 100m, memory: 256Mi }
27     limits:   { cpu: 500m, memory: 512Mi }
28
29 redis:
30   architecture: standalone
31   auth:
32     enabled: false
33
34 config:
```

```
35   nodeEnv: production
36   mongodbUri: mongodb://mongodb:27017/aiops
37   redisHost: redis-master
38   redisPort: "6379"
39   groqModel: llama-3.3-70b-versatile
40
41 ingress:
42   enabled: true
43   className: traefik
44   host: aiops.local
45
46 secrets: {}    # overridden by the git-ignored values.secrets.yaml
```

Listing A.1: Complete helm/aiops/values.yaml

Appendix B

Full Jenkinsfile

Listing B.1 reproduces the complete declarative pipeline, of which only a condensed excerpt was shown as Listing 5.8 in Chapter 5.

```
1 pipeline {
2     agent any
3     environment {
4         DOCKERHUB_TOKEN = credentials('dockerhub-token')
5         KUBECONFIG = '/var/jenkins_home/.kube/config'
6         SECRETS_FILE = '/home/vagrant/aiops-secrets/values.secrets.
            yaml'
7     }
8     stages {
9         stage('Install backend deps') {
10             steps { sh 'cd aiops-platform/backend && npm ci' }
11         }
12         stage('Test backend') {
13             steps { sh 'cd aiops-platform/backend && npm test' }
14         }
15         stage('Install frontend deps') {
16             steps { sh 'cd aiops-platform/frontend && npm ci' }
17         }
18         stage('Test frontend') {
19             steps { sh 'cd aiops-platform/frontend && npm test' }
20         }
21         stage('Build backend image') {
22             steps { sh 'docker build -t khalil512/aiops-backend:
                latest aiops-platform/backend' }
23         }
24         stage('Build frontend image') {
25             steps { sh 'docker build -t khalil512/aiops-frontend:
                latest aiops-platform/frontend' }
26         }
27         stage('Push images') {
28             steps {
29                 sh '''
30                     echo "$DOCKERHUB_TOKEN" | docker login -u
31                         khalil512 --password-stdin
32                     docker push khalil512/aiops-backend:latest
```

```
32         docker push khalil512/aiops-frontend:latest
33     '''
34 }
35 }
36 stage('Deploy to k3s') {
37     steps {
38         sh '''
39             helm upgrade aiops aiops-platform/helm/aiops \
40                 -f aiops-platform/helm/aiops/values.yaml \
41                 -f "$SECRETS_FILE" \
42                 --wait --timeout 5m
43             kubectl rollout restart deployment/aiops-
44                 backend deployment/aiops-frontend
45             kubectl rollout status deployment/aiops-backend
46                 --timeout=120s
47             kubectl rollout status deployment/aiops-
48                 frontend --timeout=120s
49         '''
50     }
51 }
```

Listing B.1: Complete aiops-platform/Jenkinsfile

Appendix C

Additional Code Listings

C.1 Health-score service: full grade mapping

```
1 function scoreToGrade(score) {  
2   if (score >= 90) return 'A';  
3   if (score >= 75) return 'B';  
4   if (score >= 60) return 'C';  
5   if (score >= 45) return 'D';  
6   return 'F';  
7 }
```

Listing C.1: scoreToGrade (complete)

C.2 Knowledge-base service: signature generation

```
1 const crypto = require('crypto');  
2  
3 function generateSignature(errorType, rootCause) {  
4   const normalized = `${errorType}:${rootCause.slice(0, 100)}`  
5     .toLowerCase()  
6     .trim();  
7   return crypto.createHash('md5').update(normalized).digest('hex');  
8 }
```

Listing C.2: generateSignature (complete)

C.3 Frontend: Axios client with automatic token refresh

```
1 apiClient.interceptors.response.use(  
2   (response) => response,  
3   async (error) => {  
4     if (error.response?.status === 401 && !error.config._retried) {  
5       error.config._retried = true;
```

```
6     const { accessToken } = await authApi.refresh();
7     store.dispatch(setAccessToken(accessToken));
8     error.config.headers.Authorization = `Bearer ${accessToken}`;
9     return apiClient(error.config);
10  }
11  return Promise.reject(error);
12 }
13 );
```

Listing C.3: Axios interceptor refreshing an expired access token

Appendix D

Glossary

Root-cause analysis

The process of identifying the underlying technical cause of an observed failure, as opposed to merely describing its symptoms.

Knowledge-base signature

A deterministic MD5 hash of an error's type and the first 100 characters of its normalized root cause, used as a cache key for previously diagnosed solutions.

Health score

A 0–100 composite indicator computed from six weighted operational metrics, mapped to a letter grade (A–F) per monitored project.

Cache hit / miss

In this platform's context, whether a given error signature is already present in the knowledge base (hit, solution reused) or not (miss, the LLM is invoked).

Webhook

An HTTP callback that an external system (here, GitLab) issues automatically when an event occurs (a pipeline finishing, a merge request opening).

BullMQ

A Redis-backed job-queue library for Node.js, used here to decouple webhook ingestion from the slower LLM-based analysis.

Helm chart

A packaged, parameterized description of a set of Kubernetes resources, installed and upgraded as a single versioned unit.

k3s

A certified, lightweight Kubernetes distribution suited to resource-constrained or edge/lab environments.

MTTR

Mean Time To Resolve: the average duration between an incident's detection and its resolution.